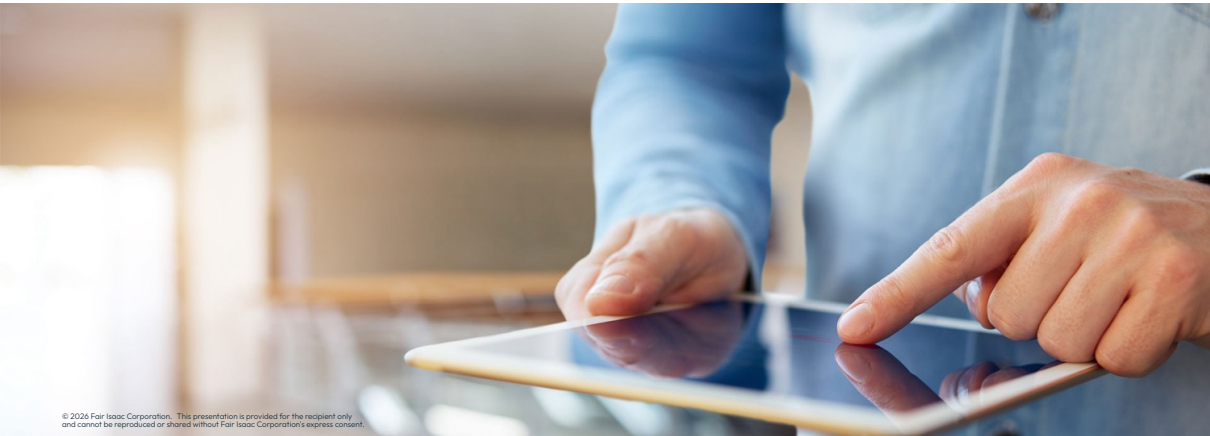
Advanced topics with FICO® Xpress Solver (Python)

FICO Xpress Training



Introduction to the course

Format

- Course split into topics, each topic comprises:
  - Introduction to concepts and recommendation of best practices
  - Running example in Python Notebook using FICO® Xpress Solver
- In this course you will learn how to:
  - Configure and tune the Xpress MIP solver for your problem
  - Diagnose and handle infeasible and numerically challenging models
  - Use advanced solver features: nonlinear solving, multiple MIP solutions, and multi-objective optimization
- Other materials:
  - This deck focuses on selected areas that are of practical importance
  - Not a replacement for the reference manuals
  - Does not try to be exhaustive ⇒ you will need to look up the details by yourself

Finding help: *Check the Xpress Python Examples and the Xpress Solver reference manual, including the Python Interface Reference Manual*

Course Overview

- Xpress overview
- **MIP solving and tuning**
- **Tolerances and scaling**
- **Infeasibility handling**
- **Nonlinear solving and tuning**
- **Multiple MIP solutions**
- **Multi-objective optimization**

Running the Python Notebook examples

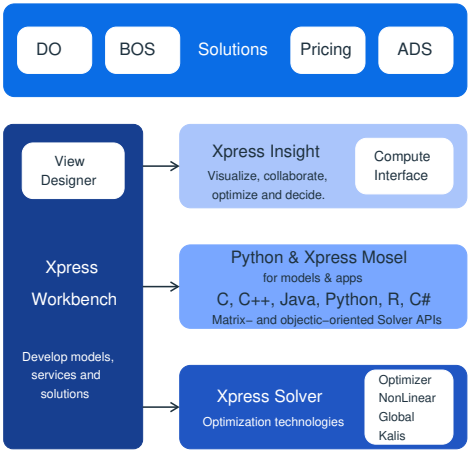
- General instructions:
  - The accompanying Python notebook contains text descriptions, mathematical formulations, and code snippets to be analyzed, run and understood in the order they appear
  - Two versions are provided: `Xpress_solver_training.ipynb` contains exercises with TODO items for you to complete; `Xpress_solver_training_Solutions.ipynb` contains completed reference examples - note that other valid solutions exist
  - *Tip: try the exercises yourself before looking at the reference solutions!*

- *Option 1*: Run on GitHub Codespaces (Recommended):
  - Run the notebook in the cloud - no installation needed! You only need a GitHub account
  - Upload the notebook to a codespace created from our python-notebooks public repo
  - Alternatively, use Google Colab or other equivalent cloud platforms

- *Option 2*: Run locally (for experienced users, when using a non-community license or working with sensitive data). Requirements:
  - Python >= 3.10, <= 3.14
  - Jupyter Notebook or other compatible IDE (VS Code, PyCharm)
  - Latest version of the following packages: *xpress, pandas, scipy, matplotlib*

FICO Xpress Optimization

The most complete optimization software on the market



Xpress Solutions

- Point-and-click applications designed for business users, includes **Decision Optimizer** and **Business Outcome Simulation**.

Xpress Technology

- **Xpress Insight** – Rapidly deploy analytic & optimization models as powerful applications.
  - **Compute Interface** for optimization execution.
- **Python or Xpress Mosel** – develop models and apps in the preferred language.
- **Xpress Solver** – Optimization algorithms and technologies to solve linear, mixed integer, non-linear, global and constraint programming problems.
- **Xpress Workbench** – IDE for developing optimization models, services, and solutions.
  - Drag & Drop view configuration using **View Designer**

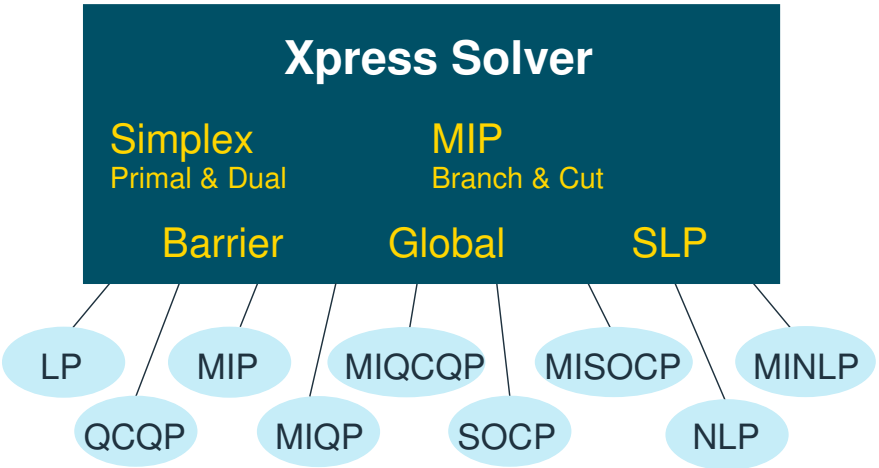
Overview of Xpress

- Optimization algorithms
  - solve different classes of problems
  - built for speed, robustness and scalability

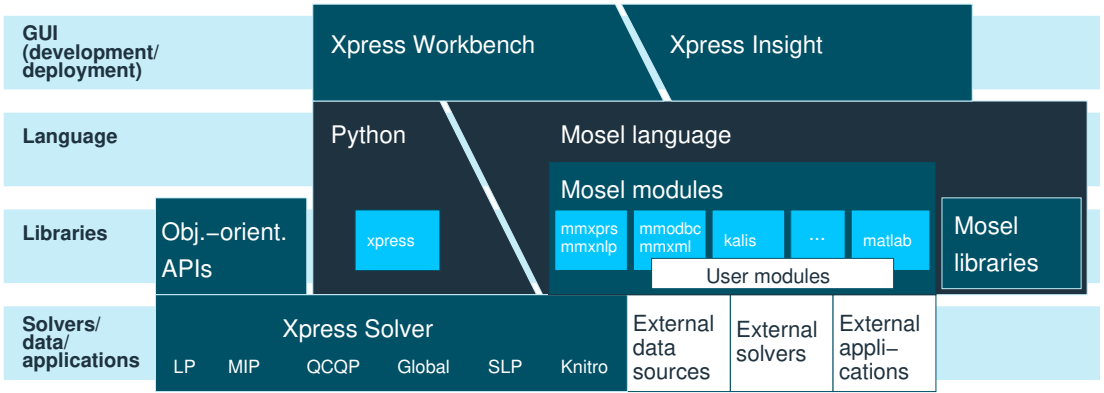
Overview of Xpress

- Optimization algorithms
  - solve different classes of problems
  - built for speed, robustness and scalability
- Modeling interfaces
  - Mosel
    - formulate model and develop optimization methods using Mosel language / environment
  - Python and object-oriented APIs
    - build up models in your application code using APIs for C, C++, Java, .NET, Python, R, Julia, MATLAB
- Application development
  - Xpress Insight
    - deploy multi-user optimization applications

Optimization algorithms



Overview of Xpress



MIP solving in Xpress

Basic concepts

- IP = Integer Programming
- MIP = Mixed Integer Programming
- MILP = Mixed Integer Linear Programming
  - All used (by us) to mean the same thing!
  - An LP in which some of the variables take integer values

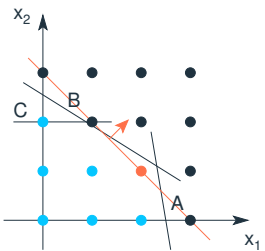
Basic concepts

- IP = Integer Programming
- MIP = Mixed Integer Programming
- MILP = Mixed Integer Linear Programming
  - All used (by us) to mean the same thing!
  - An LP in which some of the variables take integer values

- Allows wealth of decisions to be modeled:
  - Discrete choices: yes/no, on/off, etc
  - Logical conditions: if do A, must do B
  - Discrete quantities: batch sizes
  - Fixed costs
  - Price breaks
  - Piecewise linear functions

Example MIP

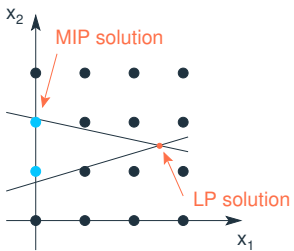
$$
\begin{array}{lrcrcl}
\max & x_1 & + & x_2 & & \\
\text{s.t.} & 6x_1 & + & x_2 & \leqslant 15 & \text{(A)} \\
 & 5x_1 & + & 8x_2 & \leqslant 20 & \text{(B)} \\
 & & & x_2 & \leqslant 2 & \text{(C)} \\
 & x_1, x_2 & \geqslant & 0 & \textbf{and integer} &
\end{array}
$$



$\Rightarrow$ Optimal MIP solution: $x_1 = 2, x_2 = 1$

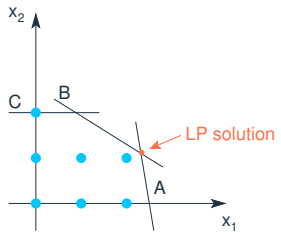
Solution techniques

- Cannot simply round the LP solution:
  - None of the grid points surrounding the 'LP solution' point lie within the feasible region
  - So rounding up or down to the closest integer values will not necessarily result in a MIP solution



LP relaxation

- Relax integer conditions and solve as an LP...



- If the LP solution happens to be integer, then it is the optimal MIP solution
- If the LP is infeasible, then the MIP is infeasible
- In both cases we have “solved“ the MIP problem
- If not... ⇒ Branch & Bound

Presolve

- Presolve attempts to simplify the problem by detecting and removing redundant constraints, tightening variable bounds, etc.
  - Applied before Branch & Bound, which then acts on the presolved problem
  - In some cases, infeasibility may be determined at this stage
- Presolve logging:

```
Original problem has:
       230 rows         2025 cols        12150 elements      1800 entities
Presolved problem has:
       210 rows         1600 cols         9600 elements      1600 entities
Presolve finished in 0 seconds
```

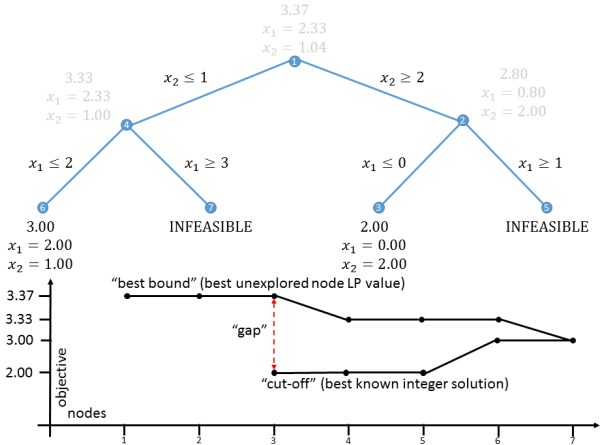
  - Control presolve is ON by default, set to 0 to turn it OFF:

```
p.controls.presolve = 0
```

  - Control the amount of probing during presolve with the preprobing control
  - Perform several rounds of presolving with presolvepasses

Branch and Bound

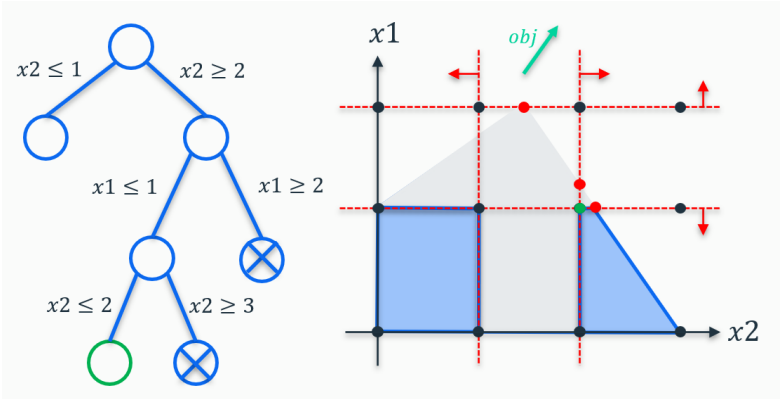
- Example: Branch and Bound for a maximization problem



FICO

Branch and Bound

- How the solver searches for the Branch and Bound tree:
  - Select a 'best node' (by default the best bound, but controllable via the backtrack control)
  - Performs a full dive in (control when to backtrack using the localchoice control)



Heuristics

- The Xpress Optimizer contains a wide variety of heuristics to help find feasible solutions during a MIP solve:
  - Simple rounding heuristics:
    - Fast heuristics that apply some simple rules to the solution of the continuous relaxation in order to produce a feasible MIP solution
    - These are typically run on every node
  - Diving heuristics:
    - Combine rounding and fixing of MIP entities and re-optimize to construct a better quality MIP solution
    - They are run frequently on both the root node and during the branch and bound tree search
    - Specific controls: heurfreq, heurdivestrategy, heurdivespeedup
  - Local search heuristics:
    - The most expensive heuristics and involve solving one or more smaller MIPs whose feasible regions describe a neighborhood around a candidate MIP solution
    - Run during the root cut-loop and typically on every 500 - 1000 nodes during the tree search
    - Specific controls in the next slides

Heuristics

- Important control to experiment with -> heuremphasis control:
  - Try different heuristic strategies that impact the Primal-Dual Integral (PDI)
  - Setting `heuremphasis=1` means more emphasis on finding good solutions early, i.e. on the primal side of the PDI:

```
p.controls.heuremphasis = 1
```

  -1: Automatic selection of heuristic (default)
  0: No heuristics
  1: Focus on reducing the primal-dual gap in the early part of the search
  2: Extremely aggressive search heuristics

Hint: *The value of the PDI can be obtained by querying the problem attribute primaldualintegral*

Large Scale Neighborhood Search (LSNS) heuristics

- Control the overall local search heuristic effort with heursearcheffort:

```
p.controls.heursearcheffort = 2
```

  - A multiplier (type `double`) on the default amount of work the local search heuristics should do
  - A higher value means the local search heuristics will be run more often and that they are allowed to search larger neighborhoods

Large Scale Neighborhood Search (LSNS) heuristics

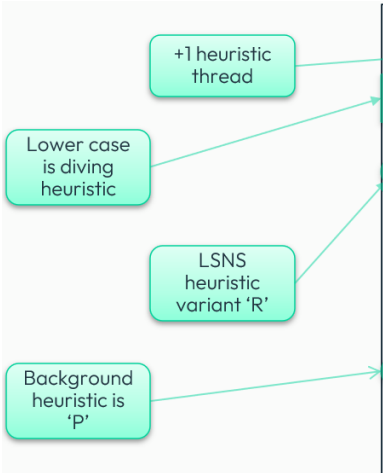
- Control the overall local search heuristic effort with heursearcheffort:

```
p.controls.heursearcheffort = 2
```

  - A multiplier (type `double`) on the default amount of work the local search heuristics should do
  - A higher value means the local search heuristics will be run more often and that they are allowed to search larger neighborhoods
- Other related controls:
  - heurfreq: specifies how many cutting rounds should be done between two runs of the local search heuristic
  - heursearchfreq: for specifying after how many nodes to run the local search again during the tree
  - heursearchrootselect: control local search heuristics to apply on the root node (bit vector)
  - heursearchtreeselect: control local search heuristics to apply during the tree search (bit vector)

Hint: *Heuristic solutions found during the tree search are marked with a letter in front of the log line. A star (*) refers to a MIP-feasible relaxation solution. For more information, check the reference manual page*

MIP Heuristic Logging



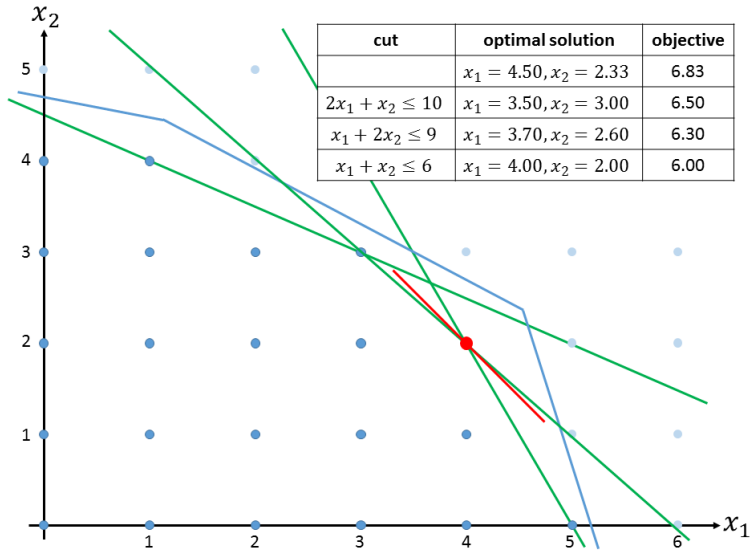

```
Starting root cutting & heuristics
Deterministic mode with up to 1 additional thread

  Its Type    BestSoln     BestBound    Sols    Add    Del     Gap     GInf    Time
a          -2485188.960 -2608070.316       3                   4.71%       0       0
q          -2573127.148 -2608070.316       4                   1.34%       0       0
R          -2573519.443 -2608070.311       5                   1.32%       0       0
R          -2574625.145 -2608070.311       6                   1.28%       0       0
R          -2575017.441 -2608070.311       7                   1.27%       0       0
R          -2586536.868 -2608070.311       8                   0.83%       0       0
R          -2586938.594 -2608070.311       9                   0.81%       0       0
R          -2589181.066 -2608070.311      10                   0.72%       0       0
R          -2589185.326 -2608070.311      11                   0.72%       0       0
   1  K    -2589185.326 -2608070.311      11     27      0     0.72%      16       0
   2  K    -2589185.326 -2608070.310      11     33     20     0.72%      35       0
   3  K    -2589185.326 -2608070.309      11     30     29     0.72%      47       0
   4  K    -2589185.326 -2608070.308      11     32     29     0.72%      45       0
   5  K    -2589185.326 -2608070.307      11     16     28     0.72%      48       0
   6  K    -2589185.326 -2608070.307      11     19     14     0.72%      54       0
   7  K    -2589185.326 -2608070.306      11     48     21     0.72%      52       0
   8  K    -2589185.326 -2608070.305      11     31     46     0.72%      56       0
P          -2590358.518 -2608070.305      12                   0.68%       0       0
P          -2591357.362 -2608070.305      13                   0.64%       0       0
   9  K    -2591357.362 -2608070.305      13     47     31     0.64%      53       0
  10  K    -2591357.362 -2608070.304      13     50     46     0.64%      54       0
P          -2591682.206 -2608070.304      14                   0.63%       0       0
P          -2591953.946 -2608070.304      15                   0.62%       0       0
```

Branch and cut

- A cut is a new constraint, which doesn't change the IP solutions, but cuts off parts of the LP relaxation
- By adding cuts, we strengthen the formulation
- This gives stronger bounds and removes the need for some branching
- But generating cuts takes time, and cuts increase the size of the problem
- So cuts can also slow down optimization

Cut example



cut	optimal solution	objective
	$x_1 = 4.50, x_2 = 2.33$	6.83
$2x_1 + x_2 \leq 10$	$x_1 = 3.50, x_2 = 3.00$	6.50
$x_1 + 2x_2 \leq 9$	$x_1 = 3.70, x_2 = 2.60$	6.30
$x_1 + x_2 \leq 6$	$x_1 = 4.00, x_2 = 2.00$	6.00

Cuts

- Try different levels of automatic cuts with the cutstrategy control:

```
p.controls.cutstrategy = 2
```

  0: cut generation disabled (always try this once)
  1: conservative cut generation strategy
  2: moderate cut generation strategy
  3: aggressive cut generation strategy
 -1: automatic choice between options 1 - 3

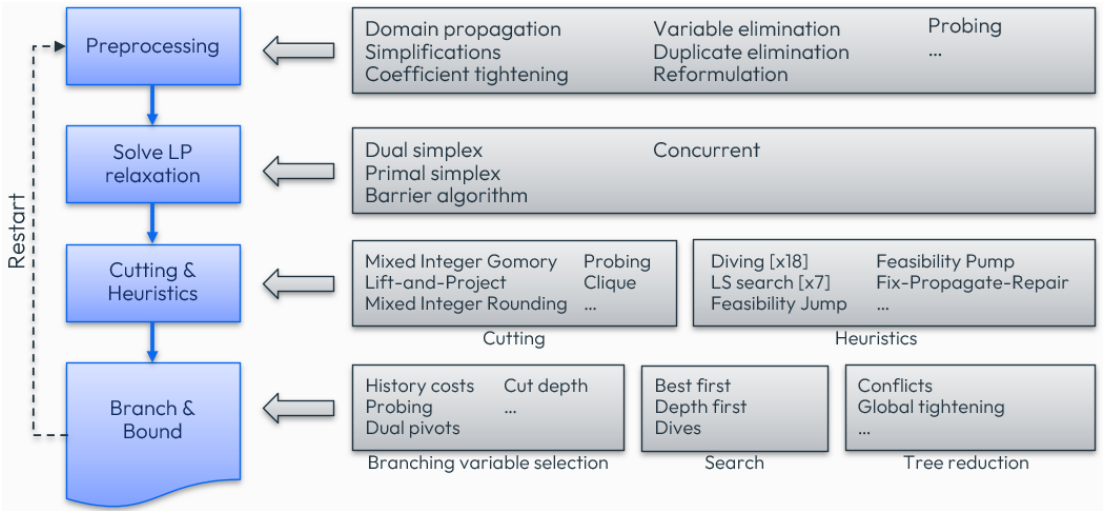
Cuts

- Other controls for determining the amount of in-tree cutting without specifying any specific cut separator:
  - gomcuts and covercuts: number of rounds of Gomory and lifted cover cuts at the root node:
    - There is usually an increased benefit from generating these at the root node, since these inequalities then apply to every subsequent node in the tree search
  - cutselect and treecutselect: number of rounds of lifted cover inequalities at the root node and tree search
    - These are bit vector controls providing detailed management of the cuts created

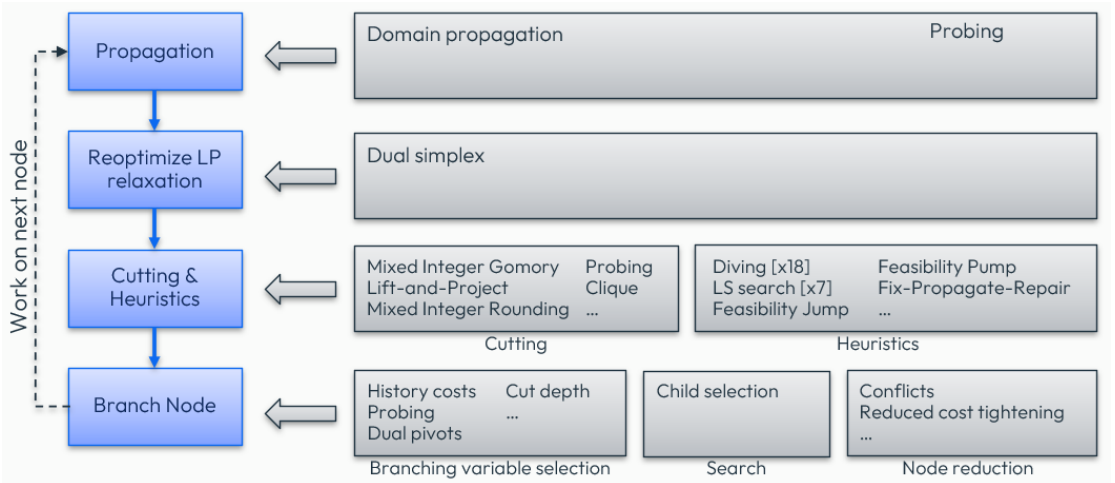
Branch and Cut - Concepts

- Preprocessing: set of routines that eliminates redundant elements and strengthens a model formulation with the aim of accelerating the subsequent solution process (includes presolving)
- Domain propagation: tighten the local domains of variables by inspecting the constraints and current domains of other variables
- Probing: technique that looks at the logical implications of fixing certain variables (*e.g.* fixing a binary variable to 0 and to 1)
- Conflict analysis: generate valid global constraints from infeasible subproblems, derived from cut off subproblems
- Strong branching: precomputes the dual bounds of potential child nodes by solving auxiliary LPs, which leads to smaller search trees

Branch and Cut - Overview



Branch and Cut - Node



Near optimal solutions

- Set stopping criteria if you only require a MIP solution close to optimal:
  - Setting a relative value stopping criterion (example for 1 percent):

```
p.controls.miprelstop = 0.01
```

$$\frac{|\,\text{MIPOBJVAL} - \text{BESTBOUND}|}{\max(|\,\text{BESTBOUND}|,|\,\text{MIPOBJVAL}|)} \leq \text{MIPRELSTOP}$$

  - Setting an absolute value stopping criterion:

```
p.controls.mipabsstop = 100
```

- Set a cut-off if you know a value the MIP solution must beat with mipabscutoff:

```
p.controls.mipabscutoff = 110.0
```

  - The closer the cut-off is to the optimal solution, the quicker the solve will be
  - Setting a cutoff can make a solve slower because the solver might have difficulties to find a first feasible solution
  - If a solution is known it is always better to load it as a start solution instead of setting a cutoff

Optimizer built-in Tuner

- The FICO® Xpress Optimizer Tuner is a tool to automate the process of discovering better control parameter settings:
  - There are over 200 controls affecting solver performance
  - Tuner systematically tests the problem against a range of different combinations of control settings
  - A single tuning run will typically involve solving each problem at least 100-200 times:
    - Can therefore become computationally very expensive for large problems

Optimizer built-in Tuner

- The FICO® Xpress Optimizer Tuner is a tool to automate the process of discovering better control parameter settings:
  - There are over 200 controls affecting solver performance
  - Tuner systematically tests the problem against a range of different combinations of control settings
  - A single tuning run will typically involve solving each problem at least 100-200 times:
    - Can therefore become computationally very expensive for large problems
- Examples of tuner-related controls and functions:

```
p.controls.tunermaxtime = 100      # set max time spent in tuning
p.controls.tunerthreads = 2        # number of concurrent tuner jobs per thread
p.tunerWriteMethod('default.xtm') # export tuner options onto an XTM
p.tunerReadMethod('default.xtm')  # read tuner options from a file
p.tune('g')                        # tune the problem as a MIP
p.optimize()               # optimize the problem with best control settings found
```

Finding help: *Check the Xpress Optimizer tuning guide to learn more about the built-in Tuner*

Warm-start for Barrier

- Barrier can be warm-started in memory using the problem.loadlpsol() method:
  - To accept the given warm-start solution, set the barstart control to -1
- By default, Barrier will use a weighted combination of its own starting point and the provided solution:
  - Use the barstartweight control to experiment with this weighting
- Any provided LP warm-start solution will be used by the Barrier solve of the initial LP relaxation of a MIP:
  - Use the 'l' flag to p.readSlxSol("sol", 'l') for providing a warm-start LP solution using an `.slx` file

Hint: *Check the online Python example that demonstrates warm-starting of Barrier, both using a file and through memory*

Other possibilities for boosting solves

- Add a starting basis with p.loadBasis():
  - Set p.controls.keepbasis=0 to prevent automatic warm-starting
- Use branching priorities with p.loadDirs() to prioritize groups of integer variables:
  - Directives can be given relating to priorities, forced branching directions, pseudo costs and model cuts
- Interact with the solver via callbacks to:
  - Add your own heuristic solutions
  - Tighten node problems
  - Add your own cutting planes / constraints
  - Provide multiple branching candidate for the Solver to choose between
  - Override the Solver branching choice
  - ...

Multi-Threading

- Single machine, shared memory multi-threading:
  - Automatic detection of logical cores available, including container restrictions, with the attribute coresdetected
  - Thread limit set automatically to match cores detected
- Main control: threads, the default number of threads used during optimization
  - Hierarchy of threading controls for fine-grained adjustments with the controls below
- What can be multi-threaded:
  - Dual simplex: use the dualthreads control
  - Barrier algorithm: barthreads and crossoverthreads
  - Concurrent LP/QP: concurrentthreads
  - MIP:
    - Root heuristics: heurthreads and backgroundmaxthreads
    - Branch-and-bound: mipthreads

Predictability and reproducibility in Xpress Solvers

- All solver components are deterministic by default:
  - Running the same problems from the same starting conditions (*e.g.* control settings) will produce identical solves
- Platform independence:
  - win64, linux64 and mac64 on *Intel* will produce identical solves
  - linux64 and mac64 on *ARM* will produce identical solves
- Minimal dependence on threads and architecture:
  - To ensure reproducibility, use the threads control to match number of threads

Note: *Determinism and reproducibility cannot be guaranteed when a user interacts non-deterministically with a solve through callbacks or when setting non-deterministic stopping criteria (e.g. time limit)*

Python Notebook [PN-1]:
Solving and tuning a MIP problem

- Navigate to Exercise 1 - Solving and tuning a MIP problem
- **Exercise 1.1**:
  - Experiment by switching the heuristic emphasis control between 0, 1, and 2 and evaluate the impact in time to optimality
- **Exercise 1.2**:
  - Deactivate cuts by using the cut strategy control and evaluate the impact in time to optimality

Tolerances and scaling

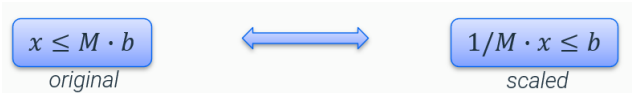
Numerical challenges

- Numerical issues arise from models with...
  - ...coefficients spanning many orders of magnitude
  - ...structures that amplify numeric error propagation
- Xpress provides tools to check coefficient ranges and condition numbers to identify numerical stability issues
- Adjust numerical tolerances, reconsider control values, and use alternative strategies like different simplex algorithms or barrier methods

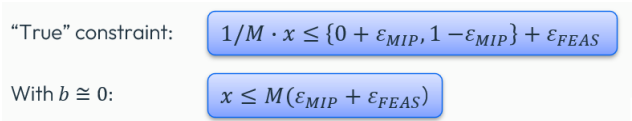
Finding help: *Check the reference manual page for more information about analyzing and handling numerical issues*

Scaling and Big-M formulations

- Example of a scaled constraint (with b binary variable)



- Valid solution: any that satisfies the (scaled) constraints within tolerances:

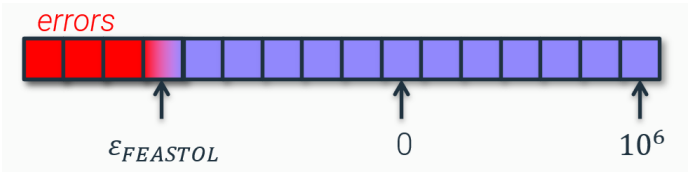


  - By default: `miptol = 5 · 10^6` and `feastol = 10^6`

- Example: $M = 10^8 \Rightarrow x \leqslant 10^8 \cdot 6 \cdot 10^{-6} = 600$ is feasible (even if $b = 0$)!

Rule of 10^6

- Double precision floating point 16-digit “budget“:



- Too large coefficients can cause round-off errors to exceed feasibility tolerance
- To stay relatively safe, use the following limits:
  - Largest value: 10^6
  - Smallest value: 10^{-6}
  - Largest row/column ratio: 10^6

Adjust numerical tolerances

- The feasibility tolerance control feastol determines when a solution is treated as feasible:

```
p.controls.feastol = 1e-6
```

  - A constraint or bound violated by less than `feastol` is treated as satisfied
- The miptol value determines when a decision variable's value is considered to be integral:

```
p.controls.miptol = 5e-6
```

  - Must be slightly larger than feastol; larger values may cause slight infeasibilities when fixing integers
- **Important:** use `feastol`/`miptol` to express the feasibility your application needs. Tightening them does not make a numerically unstable model more reliable: on ill-conditioned models it can even lead to incorrect dual bounds or false infeasibility. To address instability, use the tools on the next slides.

Scaling

- Xpress provides the user with information on the coefficient ranges in both the original problem and the problem after presolving and scaling has been applied:

```
Coefficient range                    original                    solved
  Coefficients   [min,max] : [ 1.00e+00,  8.40e+06] / [ 1.56e-01,  1.60e+01]
  RHS and bounds [min,max] : [ 1.00e+00,  8.44e+06] / [ 2.44e-02,  1.60e+01]
  Objective      [min,max] : [ 1.00e+00,  1.00e+06] / [ 4.10e+03,  2.05e+09]
Autoscaling applied Curtis-Reid scaling
```

- Autoscaling will select a scaling method that improves the coefficient distribution:

```
p.controls.autoscaling = -1
```

  - Can be disabled (0), or set to a cautious (1), moderate (2), or aggressive (3) strategy

- A more detailed scaling strategy can be defined using the bitwise scaling control:

```
p.controls.scaling = 163
```

  - If set to a non-default value by the user, autoscaling will be ignored

Diagnosing stability: the attention level

- For MIP problems, set mipkappafreq to compute the basis condition number (kappa) during the branch-and-bound search:

```
p.controls.mipkappafreq = 1
```

- The log then reports the share of bases that are stable, suspicious, unstable or ill-posed, and an overall attention level between 0 and 1:
  - 0 means all bases are stable, 1 means all are ill-posed: the higher the value, the more likely numerical errors are
  - Rule of thumb: an attention level above 0.1 should be investigated further
  - Available afterwards as the `attentionlevel` attribute (and `maxkappa` for the largest kappa sampled)
- After the root LP, Xpress also predicts the attention level from matrix features (predictedattlevel): a prediction above 0.1 logs 'High attention level predicted from matrix features'

Addressing numerical issues

- Improve the model first: narrow coefficient ranges (rule of 10^6), prefer indicators over *Big-M*
- Emphasize stability over speed with numericalemphasis (the first control to try):

```
p.controls.numericalemphasis = 1    # mild (1), medium (2), strong (3)
```

- If still hard to solve reliably, try alternative scaling, switching off dedicated presolve operations, or a different `dualstrategy`
- Refine the solution rather than loosen feasibility: the solution refiner targets higher precision after solving (see next slide)

Finding help: *See Analyzing and Handling Numerical Issues in the reference manual for the full workflow*

Solution refinement

- Xpress includes two methods for solution refinement:
  - LP Refinement (ON by default) - providing LP solutions of a higher precision:
    - iteratively attempts to increase the accuracy of the solution until tolerance targets are satisfied
  - MIP Refinement - providing MIP solutions which are truly integral:
    - attempts to round any fractional MIP entity and reduce LP infeasiblity
  - Both methods can lead to a slowdown of the solution process, which is more considerable the more numerically challenging the matrix is
- Use the refineops control to specify when the solution refiner should be executed to reduce solution infeasibilities (bit vector control):
  - The refiner will attempt to satisfy the target tolerances for all original linear constraints before presolve or scaling has been applied
- Set the precision the refiner aims for with `feastoltarget`, `miptoltarget` and `optimalitytarget`:
  - Unlike `feastol`/`miptol`, these tighten the achieved accuracy without changing what counts as feasible

Indicator constraints as alternative to *Big-M* formulations

- Indicator constraints are a very important alternative to *Big-M* formulations:
  - The solver will convert an indicator to a *Big-M* constraint if it can deduce a sufficiently small *M* that is numerically safe

- An indicator constraint is a logic constraint that expresses the implication 'if indicator condition holds then apply the constraint':
  - Represented by a tuple containing a condition on a binary variable, called the indicator, and an expression representing a constraint: `(indicator condition, constraint)`

- Indicator constraints are defined by using the problem.addIndicator() method:

```
p.addIndicator(c1, c2, ...)
```

  - Each argument `c1,c2,...` can be a single indicator constraint, or a list, tuple, or *NumPy* array of indicator constraints (tuples)
  - The constraint is only enforced when the value of the indicator variable matches a user-defined value (0 or 1)

Indicator constraints as alternative to *Big-M* formulations

- Example enforcing the constraint `y <= 15` when binary variable `x = 1` for an optimization problem `p`:

```
x = p.addVariable(vartype=xp.binary)
y = p.addVariable(lb=10, ub=20)
ind1 = (x == 1, y <= 15)
p.addIndicator(ind1)
```

Note: *The addIndicator() method also accepts nonlinear expressions for the constraint to enforce*

Python Notebook [PN-2.1]: Tolerance settings

- Navigate to Exercise 2 - Tolerances and scaling, section 2.1 (TODO 2.1)
- Run the Python code cell with the following model:

$$
\begin{aligned}
\max \quad & x_1 \\
\text{s.t.} \quad & x_1 - x_2 = 1.0000001 \\
& x_1 \leqslant 1 \\
& x_1 \geqslant 0, x_2 \geqslant 0
\end{aligned}
$$

  - Do you get the expected result?
  - Which tolerance's value would you need to change to make this work?

Python Notebook [PN-2.2]: MIP tolerance settings

- Navigate to Exercise 2 - Tolerances and scaling, section 2.2 (TODO 2.2)
- Run the Python code cell with the following model:

$$
\begin{aligned}
\min \quad & y \\
\text{s.t.} \quad & x \leqslant 1000000 \cdot y \\
& x \geqslant 1000001 \\
& y \text{ integer}
\end{aligned}
$$

  - Is the solution correct? Why ?
  - What happens after we reduce the MIP tolerance and adjust the feasibility tolerance ?

Infeasibility handling

Infeasibility

- One of the most common difficulties is what to do when a model is infeasible…
  - … is the model wrong?
  - … is the data wrong?
  - … is the problem genuinely infeasible?
  - … where is the infeasibility so I can correct or investigate it?
- Two types of infeasibility in MIP problems:
  - Continuous infeasibility: LP relaxation of the MIP problem is in itself infeasible
  - Integer infeasibility: LP relaxation is feasible but the problem is MIP infeasible
    - These are harder to detect and handle…

Detecting infeasibility with FICO® Xpress

- Infeasibility status can be identified by the *solution status* attribute after a solve:
  - The value of solstatus is `xp.SolStatus.INFEASIBLE` for an infeasible problem
  - It is returned by the p.optimize() function, together with the *solve status*:

```
solvestatus, solstatus = p.optimize()

if solstatus == xp.SolStatus.INFEASIBLE:
  print("Problem is infeasible.")
```

  - Alternatively, use the problem attribute named solstatus:

```
p.optimize()

if p.attributes.solstatus == xp.SolStatus.INFEASIBLE:
  print("Problem is infeasible.")
```

Detecting infeasibility with FICO® Xpress

- Diagnosis in presolve:
  - Possible to “trace“ back the implications that determined an inconsistency and identify a cause
  - Solver log information about infeasibility by default:
    - `The problem is infeasible due to row R67`
  - Set p.controls.trace = 1 to display a more detailed infeasibility diagnosis during presolve before calling p.lpOptimize()

Detecting infeasibility with FICO® Xpress

- Diagnosis in presolve:
  - Possible to “trace“ back the implications that determined an inconsistency and identify a cause
  - Solver log information about infeasibility by default:
    - `The problem is infeasible due to row R67`
  - Set p.controls.trace = 1 to display a more detailed infeasibility diagnosis during presolve before calling p.lpOptimize()
- Diagnosis using primal “phase 1“ simplex solution:
  - Set p.controls.presolve = -1 to force presolve to continue to simplex even when an infeasibility is discovered
  - The sum of infeasibilities is minimized in the solution
  - The resulting set of violated constraints and violated variable bounds provides a clear picture of what aspect of the model is causing the infeasibility

Finding help: *Check the reference manual page for more information about infeasibility handling, and see the diagnose_infeasible example notebook on GitHub*

Irreducible Infeasible Sets (IIS)

- Subsets of constraints, bounds and integrality conditions that form an infeasibility:
  - A model may have several infeasibilities. Repairing a single IIS may not make the model feasible
  - Works with any problem type, but does not support user functions
  - Calculating IIS can be computationally expensive, so they can be generated iteratively
  - Generation of IISs available as API functions: problem.firstIIS(), problem.nextIIS(), problem.IISAll(), problem.getIISData(), problem.writeIIS(), ...
- Basic method for LP infeasibility:
  - Filtering after first phase primal simplex
  - Uses duals of rows and reduced costs of columns to identify subproblems containing all infeasibilities

Irreducible Infeasible Sets (IIS)

- Subsets of constraints, bounds and integrality conditions that form an infeasibility:
  - A model may have several infeasibilities. Repairing a single IIS may not make the model feasible
  - Works with any problem type, but does not support user functions
  - Calculating IIS can be computationally expensive, so they can be generated iteratively
  - Generation of IISs available as API functions: problem.firstIIS(), problem.nextIIS(), problem.IISAll(), problem.getIISData(), problem.writeIIS(), ...
- Basic method for LP infeasibility:
  - Filtering after first phase primal simplex
  - Uses duals of rows and reduced costs of columns to identify subproblems containing all infeasibilities
- Basic method for MIP infeasibility when LP-feasible:
  1. Drop one or more constraints/bounds/integrality conditions of the problem
     - If the resulting problem becomes feasible, keep them
  2. Stop with an irreducible set when nothing can be removed

Irreducible Infeasible Sets (IIS)

- Generate a single IIS with p.firstIIS() or p.nextIIS(), and export it to a file using p.writeIIS():

```
if p.firstIIS(1) == 0:  # returns 0 if successful
    p.writeIIS(iis=1, filename="iis", filetype=0, flags="l")  # export in LP format
```

  - This step might likely be enough to diagnose an infeasible problem

Irreducible Infeasible Sets (IIS)

- Generate a single IIS with p.firstIIS() or p.nextIIS(), and export it to a file using p.writeIIS():

```
if p.firstIIS(1) == 0:  # returns 0 if successful
    p.writeIIS(iis=1, filename="iis", filetype=0, flags="l")  # export in LP format
```

  - This step might likely be enough to diagnose an infeasible problem
- Alternatively, use p.IISAll() to generate all IISs (only for LPs):
  - The numiis attribute to retrieve the number of IISs found so far

```
solvestatus, solstatus = p.optimize()

if solstatus.name == "INFEASIBLE":
  p.IISAll()  # generate all IIS
  numiis = p.attributes.numiis  # retrieve the number of IISs generated so far
  print(f"The problem has {numiis} IISs")
```

Irreducible Infeasible Sets (IIS)

- Use p.getIISData() to retrieve information for a specific IIS, after `p.IISAll()`, such as:
  - Size, variables (row and column vectors) and conflicting sides of the variables, duals, reduced costs, and isolation rows and columns, if available

```
for i in range(1,numiis+1):
  miisrow, miiscol, constrainttype, colbndtype, duals, rdcs, \
    isolationrows, isolationcols = [], [], [], [], [], [], [], []

 # retrieve the i'th IIS data
  p.getIISData(i,miisrow, miiscol, constrainttype, colbndtype, \
    duals, rdcs, isolationrows, isolationcols)

  print("iis data:", miisrow, miiscol, constrainttype, colbndtype, \
    duals, rdcs, isolationrows, isolationcols)
```

Irreducible Infeasible Sets (IIS)

- Perform the isolation identification procedure for an Irreducible Infeasible Set (IIS) via problem.IISIsolations(iis):
  - An IIS isolation is a special constraint or bound in an IIS
  - They indicate the likely cause of each independent infeasibility and which constraints or bounds to drop or modify
  - Removing/repairing constraints and bounds in an IIS isolation will remove all infeasibilities in the IIS without increasing the infeasibilities in other IISs
  - This procedure is computationally expensive, and is done via p.IISIsolations for an identified IIS:

```
p.IISAll()
numiis = p.attributes.numiis
for i in range(1,numiis+1):
    p.IISIsolations(i)
```

Infeasibility repair utility

- A two-phase approach to solve the problem with a minimum weighted sum of violations of relaxed constraints and bounds:
  - A set of preferred constraints and bounds is indicated, and the Optimizer attempts to identify a 'solution' that violates them minimally while satisfying all other constraints and bounds
- Available via the p.repairInfeas() method:
  - Phase I:
    - Adds *penalty* variables to constraints and bounds
    - Minimizes the weighted sum of penalties to obtain a minimally weighted infeasible solution

Infeasibility repair utility

- A two-phase approach to solve the problem with a minimum weighted sum of violations of relaxed constraints and bounds:
  - A set of preferred constraints and bounds is indicated, and the Optimizer attempts to identify a 'solution' that violates them minimally while satisfying all other constraints and bounds
- Available via the p.repairInfeas() method:
  - Phase I:
    - Adds *penalty* variables to constraints and bounds
    - Minimizes the weighted sum of penalties to obtain a minimally weighted infeasible solution
  - Phase II:
    - Adds a constraint to restrict the sum of violations to the Phase 1 solution (plus a tolerance)
    - Solves for the original objective
- More advanced functions are available to allow weights to be specified for each constraint and bound: p.repairWeightedInfeas(), p.repairWeightedInfeasBounds()

Finding help: *Check the reference manual page for more information on the repairInfeas* functions*

Infeasibility repair utility

- Example:

```
p.optimize()

if p.attributes.solstatus.name == "INFEASIBLE":

  penalty = 'c' # each penalty is the reciprocal of the corresponding preference
  phase2 = 'o'  # use the objective sense of the original problem
  flags = 'g'   # do the tree search
  delta = 0     # the relaxation multiplier in phase II

  # integer arguments represent the preferences "lepref, gepref, lbpref, ubpref"
  p.repairInfeas(penalty,phase2,flags,1,0,0,0,delta)
```

  - A preference value of 0 results in the row or bound not being relaxed, and a negative preference indicates that a quadratic penalty cost should be applied
  - The sum of violations in phase II is restricted to be no greater than `(1 + delta) * p` (with `p` = sum of violations in phase I)

Infeasibility repair utility

- p.repairWeightedInfeas() allows per-row and per-column preferences, giving precise control over which constraints can be relaxed:

```
n_rows = p.attributes.rows
n_cols = p.attributes.cols
row_types = p.getRowType(0, n_rows - 1)

# protect equality constraints (E) from relaxation, allow relaxation of others
le_prefs = [0.0 if t == 'E' else 1.0 for t in row_types]
ge_prefs = [0.0 if t == 'E' else 1.0 for t in row_types]
lb_prefs = [0.0] * n_cols  # do not relax lower bounds
ub_prefs = [0.0] * n_cols  # a choice: upper bounds could also be relaxed

p.repairWeightedInfeas(lepref=le_prefs, gepref=ge_prefs,
                       lbpref=lb_prefs, ubpref=ub_prefs)
```

Infeasibility repair utility

- Preference values for repairWeightedInfeas():
  - 0: constraint is never relaxed
  - positive value: constraint may be relaxed; higher values make relaxation cheaper (weight = 1/preference)
- ⇒ Less programming effort than manual IIS analysis, but less fine-grained and likely to give a different solution than manual constraint tightening

Avoiding infeasibility

- Try to introduce some flexibility into the problem by adding slack variables:
  - Incorporate explicit slack/surplus variables in constraints, so that if demand cannot be met or more resources are required, this is apparent from a feasible solution
  - Rather than making the model infeasible, the slack variables act as pressure valves
  - For example, replace:

$$\sum_f ship_{fc} \geqslant DEM_c \; \forall c$$

by

$$\sum_f ship_{fc} \geqslant DEM_c - xdem_c \; \forall c$$

  - Penalize the extra variables in the objective function with a high enough cost so that they are at zero unless the model in infeasible:
    - If no actual penalty costs (*e.g.* price for buying additional quantities) are known, coefficients should be one order of magnitude larger than any other coefficient...
    - ... but within the recommended numerical tolerances!

Avoiding infeasibility - Example

- **Objects:**
  - suppliers, depots, customers
- **Data:**
  - availability $AVAIL_s$, demand DEM_c, transportation cost per route: $COSTsd_{sd}$ and $COSTdc_{dc}$
- **Decisions:**
  - quantities to be shipped: $flowsd_{sd}$ and $flowdc_{dc}$
  - slack variables: $xavail_s$, $xdemand_c$
- **Constraints:**
  - remain within supply limits: $\forall s : \sum_d flowsd_{sd} \leqslant AVAIL_s + xavail_s$
  - satisfy customer demand: $\forall c : \sum_d flowdc_{dc} \geqslant DEM_c - xdemand_c$
  - flow balance in depots: $\forall d : \sum_d flowsd_{sd} = \sum_d flowdc_{dc}$
- **Objective:**
  - minimize total cost of selected routes and penalize slack: $min \sum_s \sum_d COSTsd_{sd} \cdot flowsd_{sd} + \sum_d \sum_c COSTdc_{dc} \cdot flowdc_{dc} + \sum_s PENALTY \cdot xavail_s + \sum_c PENALTY \cdot xdemand_c$

Avoiding infeasibility

- Important to keep in mind when adding slack/surplus variables:
  - Always assert the values of the added variables to ensure they are equal to zero (feasible problem)
  - If the slack variables are active (non-zero), it means that the original problem is infeasible, and you should go back and find out why
  - Be careful not to set penalty costs so large as to cause numerical issues
  - Be aware not to treat an infeasible solution as a feasible solution!

- Putting slack variables in all constraints can slow down the optimization:
  - An alternative approach is to solve the problem first without slack variables, then, if the problem is infeasible, add the slack variables to the formulation and re-solve
  - Another alternative could be to use multi-objective optimization to first find a minimally violated solution and then solve for the intended objective

Note: *Slack/surplus variables should only be introduced in inequality contraints to ensure that the model remains valid!*

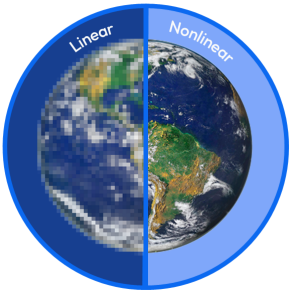
Python Notebook [PN-3]:
Infeasibility handling in Portfolio Optimization

- Navigate to Exercise 3 - Infeasibility handling in the notebook
- The portfolio model is intentionally infeasible: sector allocation limits and regional minimum requirements conflict
- The notebook walks through the full diagnostic workflow:
  1. Solve and detect infeasibility via p.attributes.solstatus
  2. Call diagnose_infeasible() helper to compute IIS and isolations
  3. Interpret the output: which constraint type is causing the infeasibility?
  4. Manual fix: relax the binding sector constraints and re-solve
  5. Automatic repair: apply p.repairWeightedInfeas() with equality constraints protected
- Both approaches yield different portfolio allocations - visualized side by side

Nonlinear Optimization (incl. FICO® Xpress Global)

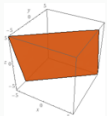
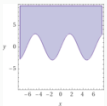
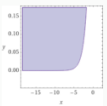
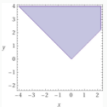
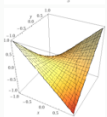
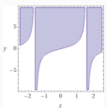
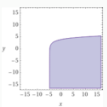
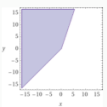
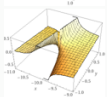
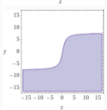
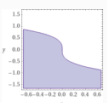
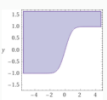
The world is nonlinear

- Many applications are inherently nonlinear:
  - Chemical processing, electrical engineering, gas transmission,...
- *A priori* linearization can work in some cases and is often quite fast...
  - ...but can be arbitrarily bad w.r.t. both feasibility and optimality.
- In many applications, guaranteed global optima are required
  - Small objective difference can equate to huge financial loss or regulatory non-compliance



What kind of nonlinear functions does FICO® Xpress support?

- All combinations of:

+, –		sin, cos		exp		| . |	
x		tan		log ln		max, min PWL	
/		arcsin arcos arctan		$\sqrt{x}$ x^y		erf erfc	

- Examples:

$$\sum_{j=1}^{n} e^{x_j} x_j^3 \qquad x_1 \sin(x_2) \qquad x \sin\left(\frac{1}{x}\right)$$

$$x_1^3 - 3x_1^2 + 3x_1 - 1 \qquad \sqrt{\sum_{j=1}^{n} x_j^2} \qquad \prod_{j=1}^{n} \log(x_j)$$

Modeling nonlinear problems in Python

- Nonlinear problems, i.e. problems containing at least one nonlinear constraint or objective, can be modeled via the Xpress Python interface:
  - Nonlinear expressions follow the same relational and arithmetic logic as linear expressions
  - Available arithmetic operators: `+`,`-`, `*`, `/`, `**` (which is the Python equivalent for the power operator, "^")
  - Univariate functions can be used from the following list: `sin`, `cos`, `tan`, `asin`, `acos`, `atan`, `exp`, `log`, `log10`,`abs`, `sign`, `sqrt`, `erf`, and `erfc`
  - The multivariate functions `min` and `max` can receive an arbitrary number of arguments
  - Piecewise linear functions (`pwl`) are discussed later

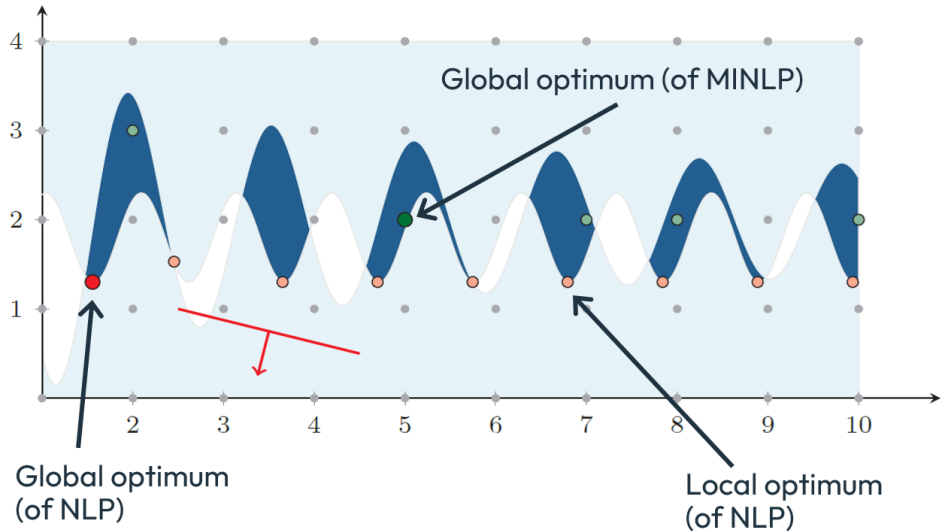
Modeling nonlinear problems in Python

- Examples of nonlinear problem elements:

```
p.addConstraint(x**4 + 2 * x**2 - 5  >= 0)   # polynomial constraint
p.addConstraint(xp.sin(math.pi * x) == 0)    # terrible way to constrain x to be integer
p.addConstraint(x**2 * xp.sign(x) <= 4)      # signum function
p.setObjective((a-x)**2 + b*(y-x**2)**2)     # minimize Rosenbrock function
```

Finding help: *For more information about modeling nonlinear problems, browse the FICO® Xpress NonLinear reference manual*

Local vs Global Optimization



Solvers

- FICO® Xpress Optimizer:
  - Used to solve convex quadratic problems (QP, MIQP, QCQP, SOCP)
- FICO® Xpress Global:
  - Finds a global optimum, including valid bounds and gaps during the solve
  - Uses spatial branching, convexification cuts, and local solver heuristics
  - Detailed techniques covered in upcoming slides
- FICO® Xpress NonLinear = Local solvers (find local optima):
  - SLP: *Successive Linear Programming*
    - Iterative linear approximations
    - Nonlinear Branch&Bound (MISLP)
  - Artelys Knitro: used as a plugin to solve highly nonlinear models. Some algorithms include:
    - Interior Point/direct (default)
    - Augmented Lagrangian
    - Nonlinear Branch&Bound
    - ...

Convex quadratic problems (QP, MIQP, MIQCQP)

- Quadratic Programming (QP): quadratic objective, linear constraints
  - Solved by Xpress Optimizer using KKT-Simplex (default) or Newton-Barrier algorithm
- Mixed Integer Quadratic Programming (MIQP): QP with integer variables
  - Solved by Xpress Optimizer using KKT-Simplex-based Branch-and-Bound (default)
  - KKT-Simplex solves QP relaxations at each node

Convex quadratic problems (QP, MIQP, MIQCQP)

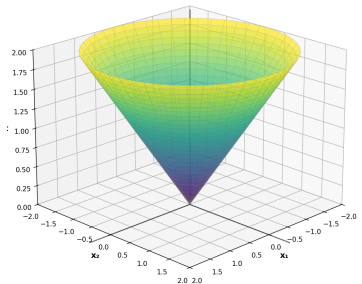
- Quadratic Programming (QP): quadratic objective, linear constraints
  - Solved by Xpress Optimizer using KKT-Simplex (default) or Newton-Barrier algorithm
- Mixed Integer Quadratic Programming (MIQP): QP with integer variables
  - Solved by Xpress Optimizer using KKT-Simplex-based Branch-and-Bound (default)
  - KKT-Simplex solves QP relaxations at each node
- Quadratically Constrained QP (QCQP): quadratic terms in constraints
  - Handled by Xpress Optimizer using Barrier-based Branch-and-Bound (default)
- Mixed Integer Quadratically Constrained QP (MIQCQP): QCQP with integer variables
  - Solved by Xpress Optimizer using Outer Approximation (default)

Convex quadratic problems (QP, MIQP, MIQCQP)

- Quadratic Programming (QP): quadratic objective, linear constraints
  - Solved by Xpress Optimizer using KKT-Simplex (default) or Newton-Barrier algorithm
- Mixed Integer Quadratic Programming (MIQP): QP with integer variables
  - Solved by Xpress Optimizer using KKT-Simplex-based Branch-and-Bound (default)
  - KKT-Simplex solves QP relaxations at each node
- Quadratically Constrained QP (QCQP): quadratic terms in constraints
  - Handled by Xpress Optimizer using Barrier-based Branch-and-Bound (default)
- Mixed Integer Quadratically Constrained QP (MIQCQP): QCQP with integer variables
  - Solved by Xpress Optimizer using Outer Approximation (default)
- **Solving these problems does not require an Xpress Nonlinear/Global solver license**

Second-order cone programming (SOCP, MISOCP)

- Second-Order Cone Programming (SOCP): Convex optimization with conic constraints
  - Special form of convex nonlinear constraints: $||Ax + b|| \leqslant c^T x + d$, linear objective function
  - Applications: Portfolio optimization, robust optimization, signal processing



- Mixed Integer SOCP (MISOCP): SOCP with integer variables
  - Common in portfolio optimization with cardinality constraints

MIQCQP and MISOCP solution approaches

- KKT-Simplex Branch-and-Bound: Primary method for convex MIQP (quadratics only in objective)
  - Solves KKT optimality conditions using Simplex-like approach
  - Handles complementarity condition (zero/non-zero variables) similar to variable bounds in Simplex

MIQCQP and MISOCP solution approaches

- KKT-Simplex Branch-and-Bound: Primary method for convex MIQP (quadratics only in objective)
  - Solves KKT optimality conditions using Simplex-like approach
  - Handles complementarity condition (zero/non-zero variables) similar to variable bounds in Simplex
- Outer Approximation: Primary method for convex MIQCQP/MISOCP
  - Approximates quadratic/conic constraints with supporting hyperplanes
  - Iteratively refines linear approximation

MIQCQP and MISOCP solution approaches

- KKT-Simplex Branch-and-Bound: Primary method for convex MIQP (quadratics only in objective)
  - Solves KKT optimality conditions using Simplex-like approach
  - Handles complementarity condition (zero/non-zero variables) similar to variable bounds in Simplex
- Outer Approximation: Primary method for convex MIQCQP/MISOCP
  - Approximates quadratic/conic constraints with supporting hyperplanes
  - Iteratively refines linear approximation
- Barrier-based Branch-and-Bound: Alternative for MIQCQP/MISOCP
  - Uses Newton-Barrier algorithm to solve continuous relaxations

MIQCQP and MISOCP solution approaches

- KKT-Simplex Branch-and-Bound: Primary method for convex MIQP (quadratics only in objective)
  - Solves KKT optimality conditions using Simplex-like approach
  - Handles complementarity condition (zero/non-zero variables) similar to variable bounds in Simplex
- Outer Approximation: Primary method for convex MIQCQP/MISOCP
  - Approximates quadratic/conic constraints with supporting hyperplanes
  - Iteratively refines linear approximation
- Barrier-based Branch-and-Bound: Alternative for MIQCQP/MISOCP
  - Uses Newton-Barrier algorithm to solve continuous relaxations
- All methods: Branch on integer variables to explore solution space

(MI)SOCP/QCQP/QP tuning

- For quadratically constrained problems (QCQP) and second-order cone programs (SOCP):
  - miqcpalg: Chooses between nonlinear Branch&Bound with barrier solves and an LP-based outer approximation
    - On most instances outer approximation performs better
    - In some cases barrier can significantly outperform it

(MI)SOCP/QCQP/QP tuning

- For quadratically constrained problems (QCQP) and second-order cone programs (SOCP):
  - miqcpalg: Chooses between nonlinear Branch&Bound with barrier solves and an LP-based outer approximation
    - On most instances outer approximation performs better
    - In some cases barrier can significantly outperform it
- Preprocessing controls for quadratic problems:
  - preconvertobjtocons: Whether to keep quadratic objective and solve with KKT-simplex or move it to a constraint to solve with barrier or OA
  - preconvertseparable: Whether to keep SCOP as is or convert to many smaller or even diagonal cones

Python Notebook [PN-4.1]: Circle packing - local vs global solver

- Navigate to Exercise 4.1 - Local vs Global NLP solver in the notebook
- The problem: pack *N* circles inside a unit square to maximize the sum of radii
- Run the local solver (SLP) code cell and note the solution quality
- Run the global solver code cell and compare:
  - Are the solutions the same?
  - What are the differences in objective value and solving time?
- Run the visualization cell to compare both solutions side by side

Xpress NonLinear vs Xpress Global

Xpress NonLinear	*Xpress Global*
Local optimum	**Global optimum**
No bounds on distance to optimum	**Duality/Optimality Gap**
No proof of infeasibility	**Can detect infeasibility, required for nonlinear IIS**
Fast in general	Possibly long branch-and-bound search
Supports user functions	No user functions

When to use FICO® Xpress Global ?

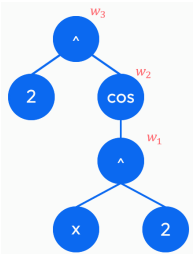
- Whenever a reliable bound on the optimal value is needed:
  - Solution quality
  - Regulatory compliance
  - Workflow validation
- Problems with many local optima
- Nonlinear problems with discrete entities
- Infeasibility detection/guarantee: IIS, repairinfeas

Xpress Global solver: High-level overview

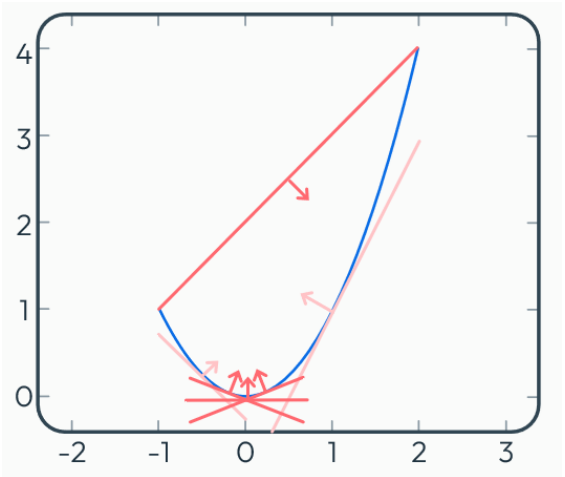
- The Global solver finds provably optimal solutions through:
  1. Nonlinear presolve: Bound tightening and problem simplification
  2. Reformulation: Transform problem using auxiliary variables (DAG representation)
  3. MIP presolve: Standard presolve on reformulated problem
  4. Root LP solve: Solve initial relaxation with convexification cuts
     - If unbounded, use bounding box approach
  5. Branch-and-bound search:
     - Additional propagation and convexification cuts to enforce nonlinearities
     - Branch on both integer and continuous variables (spatial branching)
     - Run local NLP solver on MIP-feasible solutions as a heuristic

Reformulation via Directed Acyclic Graph

- Example: $2^{cos(x^2)} \leqslant 1$
- Build the so-called expression tree
- Identify common subtrees to build a Directed Acyclic Graph
- Introduce an auxiliary variable for each internal node:
  - $w_3 \leqslant 1$ [$w_1 = x^2$, $w_2 = \cos(w_1)$, $w_3 = 2^{w_2}$]
- Identify and propagate domain reductions:
  - $w_3 \leqslant 1 \Rightarrow w_2 = \log_2(w_3) \leqslant \log_2(1) = 0$
- Ignore the nonlinear part initially and solve for the linear system



Convexification cuts



Convexification cuts

- Global solver uses convexification cuts to create linear over/underestimators:
  - Cuts are added dynamically at three stages:
    - Initial root relaxation setup
    - Root and node cutloops
    - During branching evaluation
  - Standard cuts for most operators:
    - Secant cuts: Linear interpolation between bounds
    - Tangent cuts: First-order approximations
  - Additional specialized cuts for:
    - Quadratics: Lifted tangent (LTI) cuts, RLT cuts, SDP cuts
    - Trigonometric functions: Specialized approximations
- These cuts progressively tighten the relaxation as the tree search proceeds

Spatial branching strategy

- Spatial branching extends MIP branching to continuous variables:
  - Branch on continuous variables to partition nonlinear terms' domains
  - Enables tighter convexification cuts in sub-problems

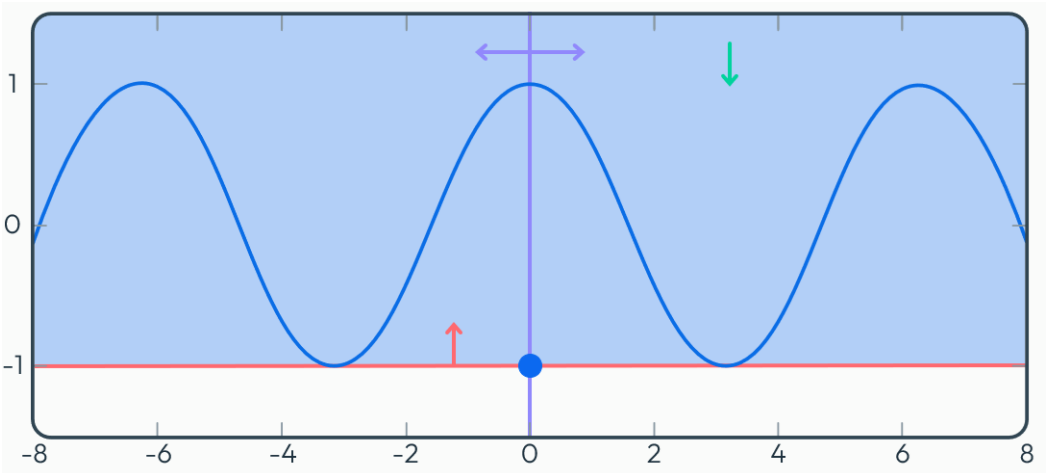
Spatial branching strategy

- Spatial branching extends MIP branching to continuous variables:
  - Branch on continuous variables to partition nonlinear terms' domains
  - Enables tighter convexification cuts in sub-problems
- Branch candidate evaluation:
  - Uses standard MIP technology: strong branching and pseudocosts
  - Propagates and cuts from candidates before evaluation
  - No inherent preference between integer and continuous variables
  - By default prefers original over auxiliary variables
    - *Exception*: When auxiliary branching leads to many bound changes at root

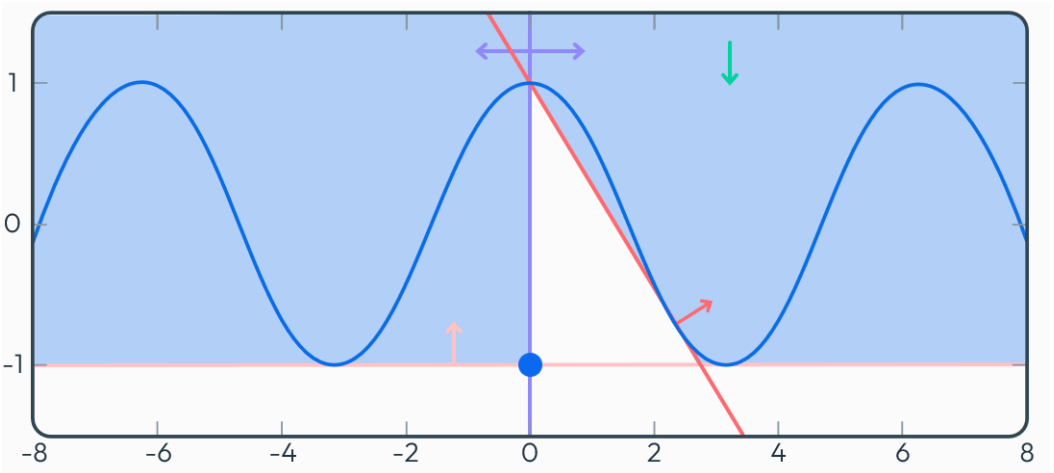
Spatial branching strategy

- Spatial branching extends MIP branching to continuous variables:
  - Branch on continuous variables to partition nonlinear terms' domains
  - Enables tighter convexification cuts in sub-problems
- Branch candidate evaluation:
  - Uses standard MIP technology: strong branching and pseudocosts
  - Propagates and cuts from candidates before evaluation
  - No inherent preference between integer and continuous variables
  - By default prefers original over auxiliary variables
    - *Exception*: When auxiliary branching leads to many bound changes at root
- Heuristics integration:
  - MIP heuristics run normally during search
  - When MIP-feasible but NLP-infeasible solution is found, local NLP solver fixes integers and solves reduced problem
  - Uses MIP-feasible solution as initial values for NLP

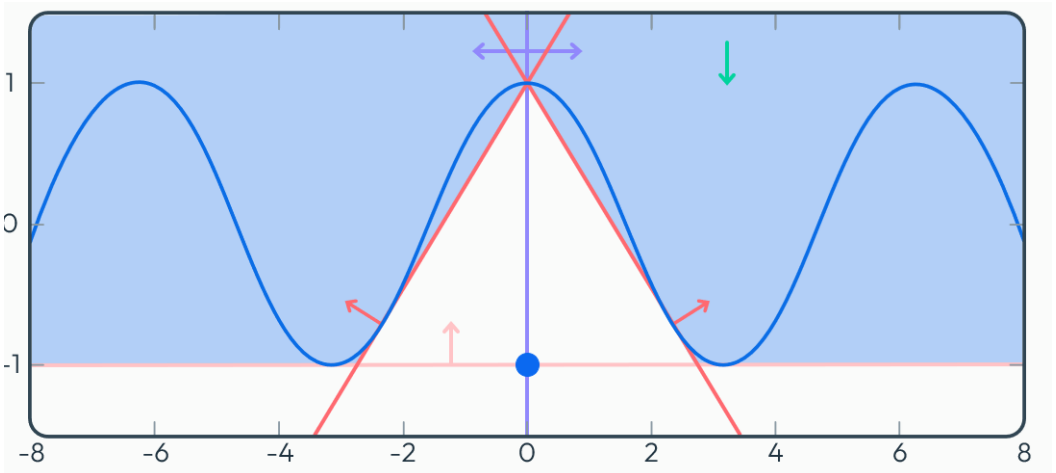
Convexification Cuts & Spatial Branching



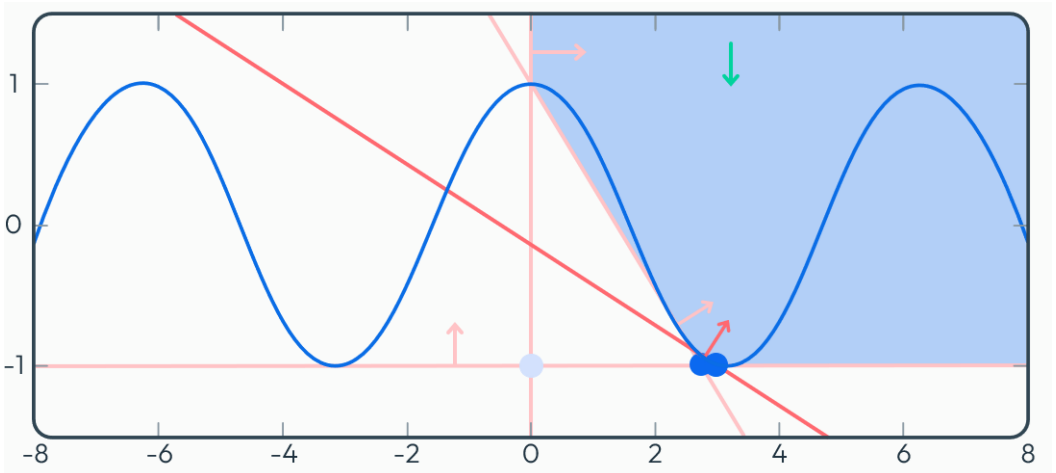
Convexification Cuts & Spatial Branching



Convexification Cuts & Spatial Branching



Convexification Cuts & Spatial Branching



Bounding Box

- LP with auxiliary variables can be unbounded:
  - min $\{x \mid x^2 - x^4 \geqslant 0\}$
- Apply initial bound propagation and convexification cuts.
- Solve the initial LP relaxation, if unbounded:
  - Impose artificial bounds on the original variables and resolve.
  - If still unbounded, impose bounds on the auxiliary variables.

Bounding Box

- If bounding box was applied:
  - Xpress Global finds (globally) best solution within those bounds
  - Better solutions may exist outside those bounds
  - Solstatus will be Feasible instead of Optimal.
- We cannot (and do not) declare global optimality in this case
- Size of bounding box can be adjusted using control globalboundingbox.
- Importance of tight bounds:
  - Xpress Global requires bounds on variables to function effectively
  - Tighter bounds on more variables lead to better performance and stronger cuts
  - Weak bounds result in weaker convexification cuts and slower solves or worse dual bounds

Nonlinear solver selection and tuning controls

- Xpress provides various controls to tune nonlinear solver performance
  - Reasonable defaults are preconfigured, but tuning can significantly improve performance for specific problem types
- Solver selection controls to choose between local and global solvers:
  - nlpsolver: Switches between local and global solves
  - localsolver: Switches between different local solvers (SLP or Knitro)

Nonlinear solver selection and tuning controls

- Xpress provides various controls to tune nonlinear solver performance
  - Reasonable defaults are preconfigured, but tuning can significantly improve performance for specific problem types
- Solver selection controls to choose between local and global solvers:
  - nlpsolver: Switches between local and global solves
  - localsolver: Switches between different local solvers (SLP or Knitro)
- Xpress Tuner supports all kinds of nonlinear solves:
  - Set tunermethod accordingly:
    - 6: SLP
    - 7: MISLP
    - 9: Global

Note: *Important caution for SLP tuning: Tuner optimizes for faster solves, which can favor settings that quickly converge to poor local optima over settings that find better solutions more slowly. Always manually evaluate solution quality vs. solve time trade-offs when using tuner with local solvers*

Global solver tuning

- Cutting plane controls for convexification:
  - globalnuminitnlpcuts: How many cuts to separate and add to the root LP for each nonlinear term
    - More cuts improve the root bound and may help with unboundedness
    - But will make LP solves more expensive
  - globalnlpcuts, globaltreenlpcuts: How many rounds of convexification cuts to add at each node before switching to branching

Global solver tuning

- Cutting plane controls for convexification:
  - globalnuminitnlpcuts: How many cuts to separate and add to the root LP for each nonlinear term
    - More cuts improve the root bound and may help with unboundedness
    - But will make LP solves more expensive
  - globalnlpcuts, globaltreenlpcuts: How many rounds of convexification cuts to add at each node before switching to branching
- Branching and propagation controls:
  - globalspatialbranchifpreferorig: Whether to always branch on original columns (1) or consider both original and auxiliary columns (0)
  - globalspatialbranchcuttingeffort, globalspatialbranchpropagationeffort: Control how much effort to spend on cutting and propagation when evaluating branching entities

Note: *Propagation and cutting on branching entities are important for good decisions but can be time and memory intensive*

Global solver tuning

- Heuristic controls:
  - globallsheurstrategy: How aggressively to run SLP on integer-feasible node solutions

Global solver tuning

- Heuristic controls:
  - globallsheurstrategy: How aggressively to run SLP on integer-feasible node solutions
- Bound tightening:
  - globalpresolveobbt: Whether to run optimization-based bound tightening at the root
    - Off by default
    - Very costly (up to two LP solves per column)
    - Can improve bounds and lead to stronger cuts

Global solver tuning

- Heuristic controls:
  - globallsheurstrategy: How aggressively to run SLP on integer-feasible node solutions
- Bound tightening:
  - globalpresolveobbt: Whether to run optimization-based bound tightening at the root
    - Off by default
    - Very costly (up to two LP solves per column)
    - Can improve bounds and lead to stronger cuts
- Example of setting global solver controls:

```
p.controls.globalnuminitnlpcuts = 10       # add 10 cuts per NL term at root
p.controls.globalnlpcuts = 5               # 5 rounds of cuts per node
```

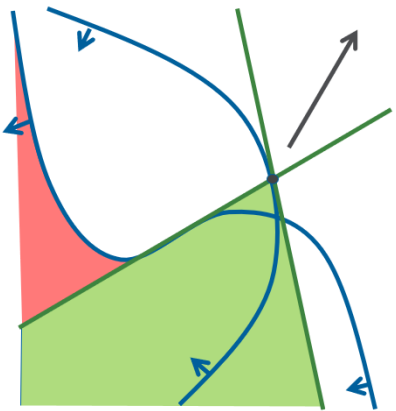
Successive Linear Programming (SLP)

- SLP algorithm: Iteratively solves linear approximations of the nonlinear problem
  - At each iteration, linearize nonlinear terms around current solution by introducing delta-variables and penalty variables :

$$\begin{aligned} &\min c^T x \\ &\text{s.t. } g_j(x) \le 0 \quad \forall j \le m \end{aligned} \qquad \begin{aligned} &\min c^T x + p^T z \\ &\text{s.t. } g_j(x_0) + \nabla g_j(x_0)^T y - z_k \le 0 \quad \forall j \le m \\ &\qquad x = x_0 + y \\ &\qquad -S \le y \le S \\ &\qquad z \ge 0 \end{aligned}$$

  - Solve the resulting LP to find a new iterate
  - Repeat until convergence (variables and objective stabilize)

Successive Linear Programming (SLP)



Create linear approximation

Might cut off feasible regions

Solve LP

Repeat

SLP: Trust regions and penalty costs

- Trust regions (step bounds): Delta-variables are step-bounded with dynamic updates
  - Ensures LP solution stays in neighborhood of current point where linearization is valid
  - Prevents unbounded linearizations when approximation is poor
  - Step bounds adapt during solve based on progress

SLP: Trust regions and penalty costs

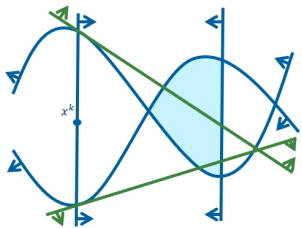
- Trust regions (step bounds): Delta-variables are step-bounded with dynamic updates
  - Ensures LP solution stays in neighborhood of current point where linearization is valid
  - Prevents unbounded linearizations when approximation is poor
  - Step bounds adapt during solve based on progress
- Penalty costs: Penalize constraint violations during iterations
  - Allows temporary infeasibility to escape poor regions
  - Fix penalty breakers if penalty terms become too large
  - Costs adjusted automatically to balance feasibility and objective improvement

SLP: Trust regions and penalty costs

- Trust regions (step bounds): Delta-variables are step-bounded with dynamic updates
  - Ensures LP solution stays in neighborhood of current point where linearization is valid
  - Prevents unbounded linearizations when approximation is poor
  - Step bounds adapt during solve based on progress
- Penalty costs: Penalize constraint violations during iterations
  - Allows temporary infeasibility to escape poor regions
  - Fix penalty breakers if penalty terms become too large
  - Costs adjusted automatically to balance feasibility and objective improvement
- Solve KKT subproblem to check for local optimality

SLP: Infeasibility

- Non-convex non-linear with feasible region.
- Iterate solution x^k in infeasible region.
- Can result in infeasible LP
- Use penalty variables to make LP feasible.



SLP convergence and local optima

- Convergence criteria: SLP stops when solution stabilizes
  - Strict convergence: Variable values don't change significantly
  - Extended convergence: Objective value and feasibility stabilize
  - Multiple tolerance levels for different convergence stages

SLP convergence and local optima

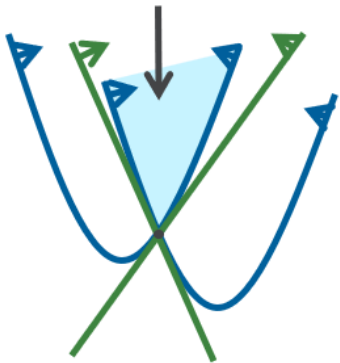
- Convergence criteria: SLP stops when solution stabilizes
  - Strict convergence: Variable values don't change significantly
  - Extended convergence: Objective value and feasibility stabilize
  - Multiple tolerance levels for different convergence stages
- Local optima behavior: SLP finds local optima, not necessarily global
  - Smaller penalty costs and larger step bounds allow exploring bigger parts of the feasible region and potentially finding better local optima at the cost of slower convergence
  - Initial point significantly affects which local optimum is found
  - For problems with many local optima, use multistart or Xpress Global

SLP convergence and local optima

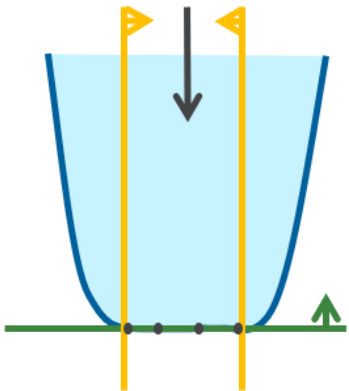
- Convergence criteria: SLP stops when solution stabilizes
  - Strict convergence: Variable values don't change significantly
  - Extended convergence: Objective value and feasibility stabilize
  - Multiple tolerance levels for different convergence stages
- Local optima behavior: SLP finds local optima, not necessarily global
  - Smaller penalty costs and larger step bounds allow exploring bigger parts of the feasible region and potentially finding better local optima at the cost of slower convergence
  - Initial point significantly affects which local optimum is found
  - For problems with many local optima, use multistart or Xpress Global
- MISLP (Mixed-Integer SLP): Combines SLP with Branch-and-Bound
  - Three approaches: Solve NLP then round, MIP outer approximation, or nonlinear Branch&Bound
  - Only guarantees local optimality for given integer solution

What convergence looks like

Strict convergence



Extended convergence



SLP tuning: Penalty costs

- Penalty costs for constraint violations during iterations:
  - slpobjtopenaltycost: Estimate initial penalty costs based on factor to objective coefficients
    - Typically values between 1 and 100
    - Smaller values lead to more iterations
    - Higher values tend to find local optima very close to the initial point
  - slperrorcostfactor: Factor by which to increase penalty costs in each iteration when rows are still infeasible
    - Typically values between 1 and 5
    - Similar effect to slpobjtopenaltycost, but applied more gradually

SLP tuning: Penalty costs

- Penalty costs for constraint violations during iterations:
  - slpobjtopenaltycost: Estimate initial penalty costs based on factor to objective coefficients
    - Typically values between 1 and 100
    - Smaller values lead to more iterations
    - Higher values tend to find local optima very close to the initial point
  - slperrorcostfactor: Factor by which to increase penalty costs in each iteration when rows are still infeasible
    - Typically values between 1 and 5
    - Similar effect to slpobjtopenaltycost, but applied more gradually
- Example:

```
p.controls.nlpsolver = 1                  # use local solver
p.controls.localsolver = 0                # use SLP
p.controls.slpobjtopenaltycost = 10       # initial penalty factor
p.controls.slperrorcostfactor = 2         # penalty increase factor
```

SLP tuning: Convergence and reformulation

- Convergence controls:
  - slpconvergenceops: What kind of convergence to require before checking for KKT conditions
    - 3: strict convergence
    - 15: extended convergence
    - 511: check KKT when not progressing any more
    - Can be useful if checking KKT conditions is particularly expensive
  - slpfilter: By default SLP returns best solution encountered; unset bit 0 to always get solution from final iterate

SLP tuning: Convergence and reformulation

- Convergence controls:
  - slpconvergenceops: What kind of convergence to require before checking for KKT conditions
    - 3: strict convergence
    - 15: extended convergence
    - 511: check KKT when not progressing any more
    - Can be useful if checking KKT conditions is particularly expensive
  - slpfilter: By default SLP returns best solution encountered; unset bit 0 to always get solution from final iterate
- Reformulation controls:
  - nlpreformulate: Decides whether `min`/`max`/`abs`/`pwl` should be handled as MIP constructs or nonlinear
    - Handling them in SLP is usually faster but finds local optima instead of global ones
    - By default converts to MIP if problem becomes fully MIP

SLP tuning: Probing and advanced options

- Probing and bound tightening:
  - nlpprobing: Apply bounds on columns and see if we can show infeasibility through propagation
    - Values from 0 to 5 with increasing levels of probing
    - Decrease if it takes too long
    - Increase if running into unboundedness

SLP tuning: Probing and advanced options

- Probing and bound tightening:
  - nlpprobing: Apply bounds on columns and see if we can show infeasibility through propagation
    - Values from 0 to 5 with increasing levels of probing
    - Decrease if it takes too long
    - Increase if running into unboundedness
- Advanced iteration controls:
  - slpdeltazlimit: For how many iterations to keep small but nonzero derivatives
    - Can be useful for avoiding bad local minima
    - Makes LP solves harder

Multistart for local solvers

- Both SLP and Knitro can use multistart with different initial points to find multiple local optima:
  - problem.msAddPreset: Create a given number of random initial points to start SLP from
    - Use `xprs.MSSET_INITIALVALUES` preset
  - problem.msAddJob: Manually add individual jobs to the multistart queue with handcrafted initial points or settings

Multistart for local solvers

- Both SLP and Knitro can use multistart with different initial points to find multiple local optima:
  - problem.msAddPreset: Create a given number of random initial points to start SLP from
    - Use `xprs.MSSET_INITIALVALUES` preset
  - problem.msAddJob: Manually add individual jobs to the multistart queue with handcrafted initial points or settings
- Example with multistart:

```
import xpress as xp

p = xp.problem()
# ... define model ...

# add 10 random initial points for multistart
p.msAddPreset(xp.constants.MSSET_INITIALVALUES, 10)

p.optimize()
```

User functions

- A user function enables the creation of an expression, the value of which is computed through external code:
  - Behaves the same as any other mathematical function, used in a formula to calculate the current value of a coefficient
  - Is normally *free-standing* and needs no access to problem or other data apart from what it receives through its argument list

User functions

- A user function enables the creation of an expression, the value of which is computed through external code:
  - Behaves the same as any other mathematical function, used in a formula to calculate the current value of a coefficient
  - Is normally *free-standing* and needs no access to problem or other data apart from what it receives through its argument list
- Use cases:
  - When an algebraic description is not possible, for example when working with a simulation or machine learning model
  - Functions that are not supported by Xpress

User functions

- A user function enables the creation of an expression, the value of which is computed through external code:
  - Behaves the same as any other mathematical function, used in a formula to calculate the current value of a coefficient
  - Is normally *free-standing* and needs no access to problem or other data apart from what it receives through its argument list
- Use cases:
  - When an algebraic description is not possible, for example when working with a simulation or machine learning model
  - Functions that are not supported by Xpress
- Important considerations when using user functions:
  - Be aware of losses in determinism and performance
  - Speed considerations: User function evaluation can dominate solve time, especially if derivatives are computed numerically

User functions

- Any user-defined function can be called within a problem by using the function xpress.user():

```
xp.user(f, a1, a2, ...)
```

  - Where `f` represents the user-defined function name and `a1, a2, ...` the necessary arguments, as in the example below:

```
def myfunc(v1, v2, data):
 model = MLmodel(v1, v2, data)    # mLmodel() defined elsewhere
 return model.results

data = readData()  # readData() defined elsewhere
x, y = p.addVariable(), p.addVariable()
p.setObjective(xp.user(myfunc, x, y, data))
```

User functions: Derivatives

- Derivative computation for user functions and model expressions:
  - Numeric derivatives: Used for user functions if derivatives not provided by the user
    - SLP queries $f(x + \varepsilon e_n)$ for all directions if derivatives not provided
    - Can become a significant bottleneck if function evaluation is costly
  - Symbolic derivatives: Default for smaller formulas, provides exact derivatives
  - Automatic differentiation: Default for larger formulas, efficient and accurate

User functions: Derivatives

- Derivative computation for user functions and model expressions:
  - Numeric derivatives: Used for user functions if derivatives not provided by the user
    - SLP queries $f(x + \varepsilon e_n)$ for all directions if derivatives not provided
    - Can become a significant bottleneck if function evaluation is costly
  - Symbolic derivatives: Default for smaller formulas, provides exact derivatives
  - Automatic differentiation: Default for larger formulas, efficient and accurate
- Providing user function derivatives can significantly improve performance:
  - Use specialized Python function types to provide derivative information
  - Many machine learning libraries in Python can provide derivatives automatically
  - See xpress.user documentation for details on providing derivatives

User functions: Derivatives

- Derivative computation for user functions and model expressions:
  - Numeric derivatives: Used for user functions if derivatives not provided by the user
    - SLP queries $f(x + \varepsilon e_n)$ for all directions if derivatives not provided
    - Can become a significant bottleneck if function evaluation is costly
  - Symbolic derivatives: Default for smaller formulas, provides exact derivatives
  - Automatic differentiation: Default for larger formulas, efficient and accurate
- Providing user function derivatives can significantly improve performance:
  - Use specialized Python function types to provide derivative information
  - Many machine learning libraries in Python can provide derivatives automatically
  - See xpress.user documentation for details on providing derivatives
- Control for user function evaluation:
  - xslpevaluate: By default (0) SLP tries to use derivatives to avoid costly re-evaluations if the new iterate is sufficiently close
    - Set to 1 to force reevaluation if userfunction is discontinuous

MISLP tuning

- MISLP (Mixed-Integer SLP) solves problems with both nonlinear and integer variables using three approaches:
  - SLP-in-MIP (default): Nonlinear Branch&Bound with SLP solve in every node - most reliable
  - SLP-MIP-SLP: Relax integers, solve SLP, fix linearization and solve MIP, fix integers and resolve SLP - fastest but risky
  - MIP-in-SLP: Single SLP solve with linearizations as MIPs - faster when nonlinear part dominates
- slpmipalgorithm: Controls which MISLP algorithm to use:
  - Set bit 9: run MIP-in-SLP (usually not recommended)
  - Set bit 10: run SLP-MIP-SLP (fastest but most likely to run into local infeasibility or bad local optima)

MISLP tuning

- MISLP (Mixed-Integer SLP) solves problems with both nonlinear and integer variables using three approaches:
  - SLP-in-MIP (default): Nonlinear Branch&Bound with SLP solve in every node - most reliable
  - SLP-MIP-SLP: Relax integers, solve SLP, fix linearization and solve MIP, fix integers and resolve SLP - fastest but risky
  - MIP-in-SLP: Single SLP solve with linearizations as MIPs - faster when nonlinear part dominates
- slpmipalgorithm: Controls which MISLP algorithm to use:
  - Set bit 9: run MIP-in-SLP (usually not recommended)
  - Set bit 10: run SLP-MIP-SLP (fastest but most likely to run into local infeasibility or bad local optima)
- Cutting planes and heuristics:
  - slpcutstrategy: Set to -1/1/2/3 to enable MIP-cutting planes in MISLP
    - Generally speeds up the solve
    - May lead to local infeasibility by cutting off bad local optima
  - slpheurstrategy: Controls heuristics in MISLP
    - When left at default, also takes heuremphasis into account

Guidelines for solver choice and tuning

- When to use Global solver:
  - Whenever a reliable bound on the optimal value is needed
  - Problems with many local optima
  - Infeasibility detection or repair

Guidelines for solver choice and tuning

- When to use Global solver:
  - Whenever a reliable bound on the optimal value is needed
  - Problems with many local optima
  - Infeasibility detection or repair
- When to use SLP over Global:
  - When quick solves are crucial (Global can be orders of magnitude slower, especially for NLPs)
  - Large-scale problems where Global's branch-and-bound becomes prohibitive
  - Problems with user functions (not supported by Global)

Guidelines for solver choice and tuning

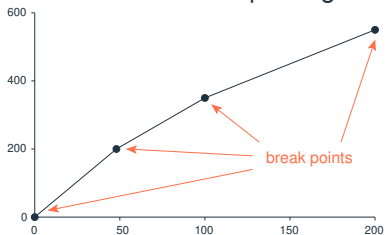
- When to use Global solver:
  - Whenever a reliable bound on the optimal value is needed
  - Problems with many local optima
  - Infeasibility detection or repair
- When to use SLP over Global:
  - When quick solves are crucial (Global can be orders of magnitude slower, especially for NLPs)
  - Large-scale problems where Global's branch-and-bound becomes prohibitive
  - Problems with user functions (not supported by Global)
- When to use Knitro:
  - Highly nonlinear models
  - When SLP convergence is slow

Guidelines for solver choice and tuning

- When to use Global solver:
  - Whenever a reliable bound on the optimal value is needed
  - Problems with many local optima
  - Infeasibility detection or repair
- When to use SLP over Global:
  - When quick solves are crucial (Global can be orders of magnitude slower, especially for NLPs)
  - Large-scale problems where Global's branch-and-bound becomes prohibitive
  - Problems with user functions (not supported by Global)
- When to use Knitro:
  - Highly nonlinear models
  - When SLP convergence is slow
- General tuning strategy:
  - Start with default settings
  - Use Xpress Tuner to automatically find good parameter settings (be careful with tuner for SLP/local solves - may optimize for speed over solution quality)
  - Fine-tune specific controls based on problem characteristics

Piecewise linear (PWL) functions

- Piecewise linear constraints define a variable as a piecewise linear function of another variable:
  - Can be used to model stepwise functions or to approximate nonlinear functions
  - Example for discounts on unit costs depending on the quantity of items bought:



- First 50 items: $COST_1 = \$4$ each
- Next 50 items: $COST_2 = \$3$ each
- Then, up to 200: $COST_3 = \$2$ each

- Quantity break points x_i: 0, 50, 100, 200
- Cost break points y_i (= total cost of buying quantity x_i): 0, 200, 350, 550

$$y_i = COST_i \cdot (x_i - x_{i-1}) + y_{i-1} \text{ for } i = 1, 2, 3$$

Piecewise linear (PWL) functions

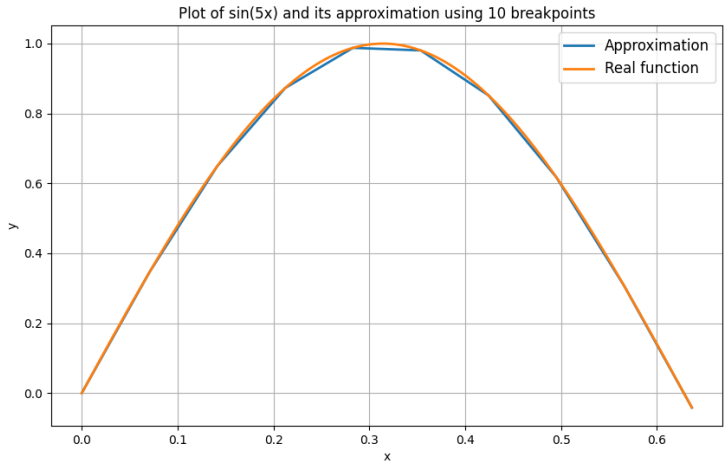
- Piecewise linear functions can be intuitively added to a problem by using the xp.pwl(`dict`) method in constraints or objectives:
  - Receives a dictionary as argument that associates intervals with linear functions with:
    - Keys: intervals specified as two-element tuples (must be pairwise disjoint, no overlap)
    - Values: linear expressions (or constants). Function must use only one variable in all of the dictionary's values
  - Modeling the previous example where `y` is a piecewise linear function of `x`:

```
x = p.addVariable(vartype=xp.integer, ub=200)
y = p.addVariable()
p.addConstraint(xp.pwl({(0, 50): 4*x,
                        (50, 100): 3*(x-50) + 200,
                        (100, 200): 2*(x-100) + 350]}) == y)
```

Note: *The piecewise linear function is always univariate, i.e. there must always be only one input variable*

Piecewise linear approximation of nonlinear functions

- Piecewise linear constraints can also be used to approximate nonlinear functions:
  - Approximating the function $sin(5\pi)$, $\forall x$ in $[0, 2/\pi]$, to be maximized:



Piecewise linear approximation using p.addpwlcons

- The low-level function problem.addPWLCons can be used for creating and directly adding piecewise linear constraints to a problem:

```
problem.addPWLCons(colind, resultant, start, xval, yval)
```

  - `colind`: integer array containing the input variables `x` of the piecewise linear functions
  - `resultant`: integer array containing the output variables `y` of the piecewise linear functions
  - *`start`*: integer array containing the start index of each constraint in the `xval` and `yval` arrays
  - *`xval`*: array containing the x-values of the breakpoints
  - *`yval`*: array containing the y-values of the breakpoints

Note: *A PWL constraint* `y = f(x)` *consists of an (input) column* `x`*, a resultant (output) column* `y` *and a piecewise linear function* `f` *described by breakpoints* `(xval,yval)`*. The function does not need to be continuous - if there are jumps, the solver may return values at any point in the jump*

Piecewise linear approximation using p.addpwlcons

1. Define breakpoints and values at breakpoints:

```
N = 10        # number of segments
freq = 5      # frequency
step = (2 / math.pi) / (N - 1)  # width of each segment = domain / (N-1)
breakpoints = np.array([i * step for i in range(N)]) # breakpoints x
values = np.sin(freq * breakpoints)                  # value at breakpoints
```

2. Define piecewise linear function using p.addPWLCons:

```
y = p.addVariable()          # auxiliary variable for 'resultant'

p.addPWLCons([x], [y], [0], breakpoints, values)
```

3. Set y as the objective function to be maximized, and optimize:

```
p.setObjective(y, xp.ObjSense.MAXIMIZE)

p.optimize()
```

Piecewise linear approximation using xp.pwl

1. Define breakpoints, values at breakpoints and slopes:

```
N = 10        # number of segments
freq = 5      # frequency
step = (2 / math.pi) / (N - 1)  # width of each segment = domain / (N-1)
breakpoints = np.array([i * step for i in range(N)]) # breakpoints x
values = np.sin(freq * breakpoints)                  # value at breakpoints
slopes = freq * np.cos(freq * breakpoints)           # derivatives
```

2. Define piecewise linear function using list comprehension with xp.pwl:

```
pw = xp.pwl((breakpoints[i], breakpoints[i+1]):
    values[i] + slopes[i] * (x - breakpoints[i]) for i in range(N - 1))
```

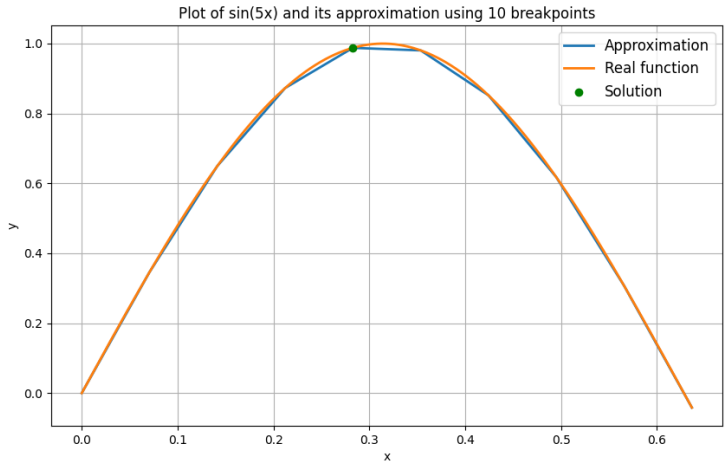
3. Set pw as the objective function to be maximized, and optimize:

```
p.setObjective(pw, xp.ObjSense.MAXIMIZE)

p.optimize()
```

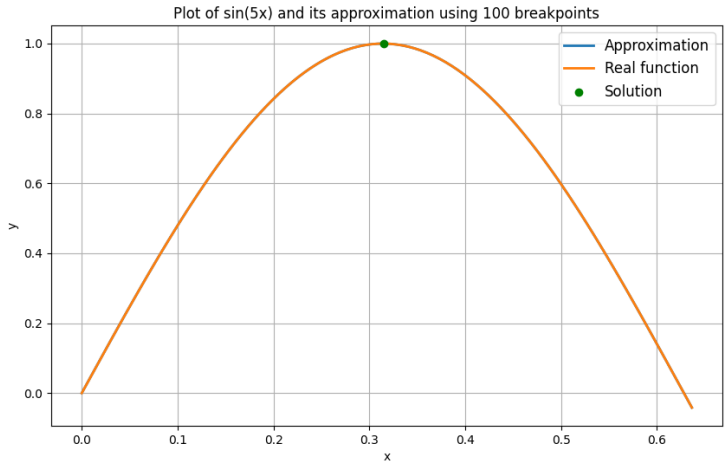
Piecewise linear approximation

- Solution for N = 10:



Piecewise linear approximation

- Solution for N = 100:



Python Notebook [PN-4.2]: Using PWL to approximate a nonlinear function

- Navigate to Exercise 4.2 - Using PWL to approximate a nonlinear function
- Run the code cells in Exercise 4.2 for progressively higher number of breakpoints *N*: 10, 15, 20, ...
- Visualize the evolution of the piecewise linear approximation and the approximation of the nonlinear function

Multiple MIP solutions

Multiple MIP solutions

- When solving a MIP problem, FICO® Xpress will typically find a number of MIP solutions before finding the optimal solution:
  - Sometimes you may want to use these intermediate solutions, or give the user the opportunity to stop the search early and use one of them
- Once it has found a MIP solution, it will only search for better MIP solutions:
  - The value of MIP solutions saved in the search always improves in the order that they are found
- Finding multiple MIP solutions is the most common use case of library callbacks

Note: *Xpress is designed to find good MIP solutions. It cannot be used to find all possible MIP solutions to a problem*

Callback functions

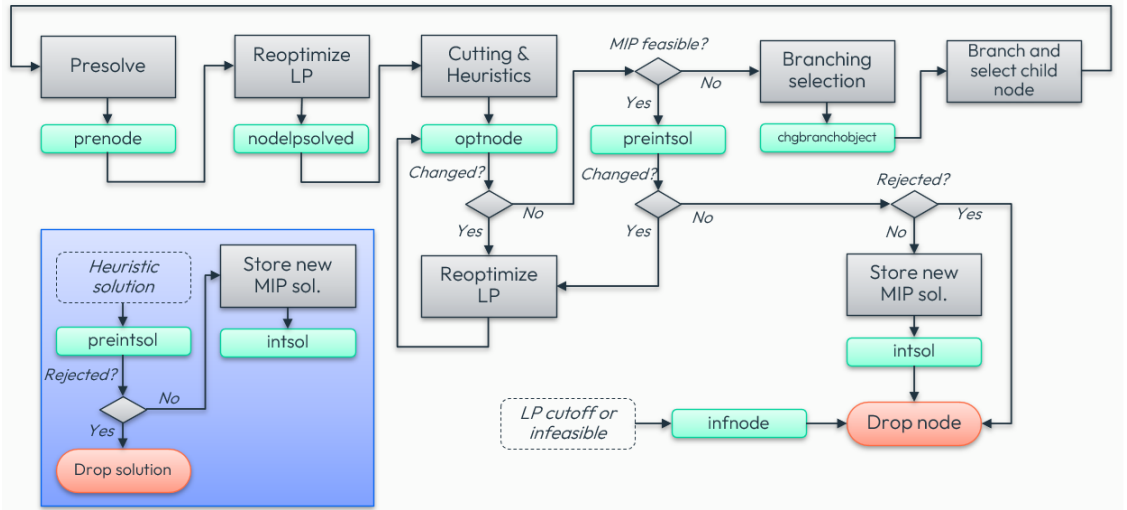
- To capture all MIP solutions found by Xpress, you need to declare a callback function
- The library callbacks are a collection of functions which allow user–defined routines to be specified to Xpress Optimizer:
  - Called at various stages during the optimization process, prompting the Optimizer to return to the user's program before continuing with the solution algorithm
  - In Python, names of functions for defining callbacks are of the form problem.add*Callback()
- With the MIP solution callback method problem.addIntsolCallback(), you can access any new integer solution found, display it, store it, output it, allow the user to examine it and stop the search early, *etc.*

Main MIP-related callbacks

- Logging callbacks: message, lplog, barlog, cutlog, miplog
- Tracking the MIP branch-and-bound tree: newnode, infnode, nodecutoff
- Callbacks that influence the MIP search:
  - prenode, nodelpsolved: tighten bounds, add solutions
  - optnode: tighten bounds, add solutions, add cuts, add branching candidates
  - chgbranchobject: override Xpress selected branching object
  - preintsol: tighten bounds, add solutions, add cuts, add branching candidates, reject MIP solution, adjust cutoff
- Controls can be changed deterministically within node specific callbacks
- Use problem.getCallbackSolution() inside the defined function to retrieve the context specific solution

Note: *Adding cuts or branching candidates is not yet supported for Global solves*

MIP Node Callback Flow



Using callback functions in Python

- Steps for using callbacks using the Python API:
  1. Define a callback function (say `exportsol`) that is to be run at certain points in time (for p.addIntsolCallback(), every time a new integer solution is found):

```
def exportsol(prob, data, soltype, cutoff):
   with open('solutions.txt', 'a') as file:
      file.write(f"New solution: {prob.getCallbackSolution()}")
```

  2. Call the corresponding problem.add*Callback() method with `exportsol` as its argument

```
p.addIntsolCallback(exportsol, data)   # assume data defined elsewhere
```

  3. Run the p.optimize() command that launches the appropriate solver

Using callback functions in Python

- Steps for using callbacks using the Python API:
  1. Define a callback function (say `exportsol`) that is to be run at certain points in time (for p.addIntsolCallback(), every time a new integer solution is found):

```
def exportsol(prob, data, soltype, cutoff):
   with open('solutions.txt', 'a') as file:
      file.write(f"New solution: {prob.getCallbackSolution()}")
```

  2. Call the corresponding problem.add*Callback() method with `exportsol` as its argument

```
p.addIntsolCallback(exportsol, data)  # assume data defined elsewhere
```

  3. Run the p.optimize() command that launches the appropriate solver

- A callback function is passed once as an argument and used possibly many times while a solver is running, and receives:
  - A `problem object` declared with `p = xp.problem()`
  - A user-defined `data object` to read and/or modify information within the callback
    - The data object may be optional, depending on the callback type

MIP solution callback

- Example for a callback function named `intsolcb` that is called every time a new integer solution is found via the p.addIntsolCallback() method:

```
import xpress as xp

def printsolcb(prob, data, soltype, cutoff):
  # callback to be used when an integer solution is found defined here
   print(f"Solution: {prob.getCallbackSolution()}")

p = xp.problem()
p.read('myprob.lp')  # reads in a problem, let's say a MIP

p.addIntsolCallback(printsolcb, data)  # assume 'data' defined elsewhere
p.optimize()
```

Note: *While the* `function` *argument is necessary for all p.add*Callback() functions, the* `data` *object can be specified as* `None`*. In that case, the callback function will be run with* `None` *as its data argument*

Finding multiple MIP solutions

- Xpress Optimizer finds increasingly better solutions:
  - New solutions that are valued the same or worse than the current best are discarded by the Optimizer by default
  - The value of the mipaddcutoff control parameter can be added to the current best solution's objective value to set a new cutoff
  - Since it is an absolute value, the sign must be set depending on the objective sense. Example for a maximization problem:

```
p.controls.mipaddcutoff = -0.5 # accept solutions 0.5 worse than current cutoff
```

- Setting mipaddcutoff to a negative (positive) value for a maximization (minimization) problem allows the solver to find worse solutions

Adding MIP solutions

- Provide Xpress Optimizer with a new feasible, infeasible or partial MIP solution via the problem.addMipSol() method:

```
p.addMipSol(solval, colind, name)
```

  - If a partial solution is provided:
    - if values are given for all discrete columns, these will be fixed to those values and the continuous part optimized
    - if some integer values are not defined, a local search will be run in an attempt to find any remaining integer feasible values
  - If the provided solution is found to be infeasible:
    - local search heuristic will be run in an attempt to find a close feasible integer solution
  - The method can also be called from callbacks

Hint: *Check the online TSP example that demonstrates the basic use of providing a full, feasible solution to warm-start a MIP solve*

Python Notebook [PN-5]: Multiple MIP solutions

- Navigate to Exercise 5 - Multiple MIP solutions in the notebook

1. Storing multiple MIP solutions via callbacks:
   - Examine the callback functions printsol, exportsol, validatesol to understand what they are programmed to do
   - Run the code cell with the optimization model three times calling a different callback function each time and analyze the output(s)
   - In the last code cell of this part, set the mipaddcutoff control value to `-0.5` and re-run the MIP model using `printsol` as callback:
     - How many additional solutions are found by the Optimizer ?
2. Adding MIP solutions:
   - Run the code cell in part 5.2 and analyze the output log:
     - Has any of the randomly generated solutions resulted feasible ?
     - If not, has a feasible solution been obtained after applying local search to any of those initially infeasible solutions ?

Optimizing with multiple objectives

Optimizing for different objectives sequentially

- The problem.setObjective() method allows users to add several linear objectives for solving a problem for different objectives sequentially:
  - Multiple calls to p.setObjective() are allowed
  - The user must define the `objidx` argument (with consecutive integers starting with 0) to indicate the multi-objective context and the sequence of objectives to consider
  - The model is run for each objective sequentially, thus runs are independent of each other
  - The `sense` of the the first objective (`objidx=0`) defines the default optimization sense for all objectives:
    - To reverse the optimization sense for secondary objectives, set the `weight` attribute to `-1`

Optimizing for different objectives sequentially

- The problem.setObjective() method allows users to add several linear objectives for solving a problem for different objectives sequentially:
  - Multiple calls to p.setObjective() are allowed
  - The user must define the `objidx` argument (with consecutive integers starting with 0) to indicate the multi-objective context and the sequence of objectives to consider
  - The model is run for each objective sequentially, thus runs are independent of each other
  - The `sense` of the the first objective (`objidx=0`) defines the default optimization sense for all objectives:
    - To reverse the optimization sense for secondary objectives, set the `weight` attribute to `-1`

```
p.setObjective(x1, objidx=0)              # minimize first objective
p.setObjective(x2, objidx=1, weight=-1)   # maximize second objective
p.setObjective(...)                       # other objectives

p.optimize()
```

  - The Optimizer will print the logs for each sequential run and, in the end, a summary of the objective values found for each run:
    - This can be useful to assess the maximum possible value for each objective

Optimizing with multiple objectives

- The problem.addObjective() method allows users to add one or more linear objectives for solving multi-objective optimization problems:
  - Use p.addObjective(), possibly after an initial call to p.setObjective(), to create additional objectives (existing objectives will remain in the same problem):

```
p.addObjective(obj1,obj2,...,priority=None,weight=None,abstol=None,reltol=None)
```

Optimizing with multiple objectives

- The problem.addObjective() method allows users to add one or more linear objectives for solving multi-objective optimization problems:
  - Use p.addObjective(), possibly after an initial call to p.setObjective(), to create additional objectives (existing objectives will remain in the same problem):

```
p.addObjective(obj1,obj2,...,priority=None,weight=None,abstol=None,reltol=None)
```

  - With at least one objective expression and a set of *optional* arguments:
    - `obj1,obj2,...`: expression(s) for the objective(s) to be added to the problem
    - *priority*: priority for the new objective(s)
    - *weight*: weight for the new objective(s); negative values invert the sense of the objective
    - *abstol*: absolute tolerance for the new objective(s)
    - *reltol*: relative tolerance for the new objective(s)

Note: *A multi-objective problem may only contain linear objective functions. The problem itself may be of any kind supported by Xpress, but nonlinear objective terms must be modeled using a transfer variable*

Optimizing with multiple objectives

- Approaches followed by the Optimizer for solving multi-objective problems:
  - Blended (or Archimedian) approach:
    - Applied when objectives have equal priority (their weights may be equal or different)
    - Weighted sum optimization, setting as objective function the linear combination of the added objectives and their weights (weights default to 1 if left undefined, giving an equally-weighted blend)

Optimizing with multiple objectives

- Approaches followed by the Optimizer for solving multi-objective problems:
  - Blended (or Archimedian) approach:
    - Applied when objectives have equal priority (their weights may be equal or different)
    - Weighted sum optimization, setting as objective function the linear combination of the added objectives and their weights (weights default to 1 if left undefined, giving an equally-weighted blend)
  - Lexicographic (or preemptive) approach:
    - Applied when each objective has a different priority and a unit weight
    - Xpress will solve the problem once for each distinct objective priority that is defined
    - All objectives from previous iterations are fixed to their optimal values within the tolerances specified in the configuration of the individual objective:

```
objective <= optimal_value * (1 + reltol) + abstol  # for minimization obj.
objective >= optimal_value * (1 - reltol) - abstol  # for maximization obj.
```

Optimizing with multiple objectives

- Approaches followed by the Optimizer for solving multi-objective problems:
  - Blended (or Archimedian) approach:
    - Applied when objectives have equal priority (their weights may be equal or different)
    - Weighted sum optimization, setting as objective function the linear combination of the added objectives and their weights (weights default to 1 if left undefined, giving an equally-weighted blend)
  - Lexicographic (or preemptive) approach:
    - Applied when each objective has a different priority and a unit weight
    - Xpress will solve the problem once for each distinct objective priority that is defined
    - All objectives from previous iterations are fixed to their optimal values within the tolerances specified in the configuration of the individual objective:

```
objective <= optimal_value * (1 + reltol) + abstol  # for minimization obj.
objective >= optimal_value * (1 - reltol) - abstol  # for maximization obj.
```

  - Hybrid approach:
    - Applied when objectives have both different priorities and different weights
    - Xpress will solve the problem once for each distinct objective priority defined, optimizing in each iteration a linear combination of the objective functions with the same priority

Optimizing with multiple objectives

- Examples:

```
# blended (weighted sum) approach: equal priority, different weights
p.addObjective(2*x + y, priority=0, weight=-0.7) # maximize, higher weight
p.addObjective(y, priority=0, weight=0.3)        # minimize, lower weight

# lexicographic approach with setObjective()
p.setObjective(xp.Dot(x, return), sense=xp.ObjSense.MAXIMIZE, priority=1) # max. return
p.addObjective(variance, priority=0, weight=-1)                  # minimize risk

# hybrid approach with three objectives
p.addObjective(xp.Sum(x), priority=1, weight=0.5, reltol=0.1)
p.addObjective(xp.Dot(A,x), priority=1, weight=0.3)
p.addObjective(xp.Dot(B,x), priority=0, weight=-0.2)
```

Hint: *Check the MULTIOBJOPS control to configure the behaviour of the optimizer when solving multi-objective problems*

Python Notebook [PN-6]: Optimizing with multiple objectives

- Navigate to Exercise 6 - Multi-objective optimization in the notebook

1. Blended (or Archimedean) and Lexicographic approaches:
   - Analyze the code in the first code cell, run it, and visualize the generated efficient frontier
   - Read the second code cell and run it. Check the solver logs and confirm that:
     - The Optimizer performs two solving procedures, printing the solver log for each one sequentially
     - The value for each objective is printed in the end of all runs
     - The resulting solution falls within the generated efficient frontier
2. *Optional: Goal programming*
   - Read the problem description to become familiar with the model
   - Run the code cell and analyze the log. Which target is not achieved by the final solution ?

Thank You

www.fico.com/optimization